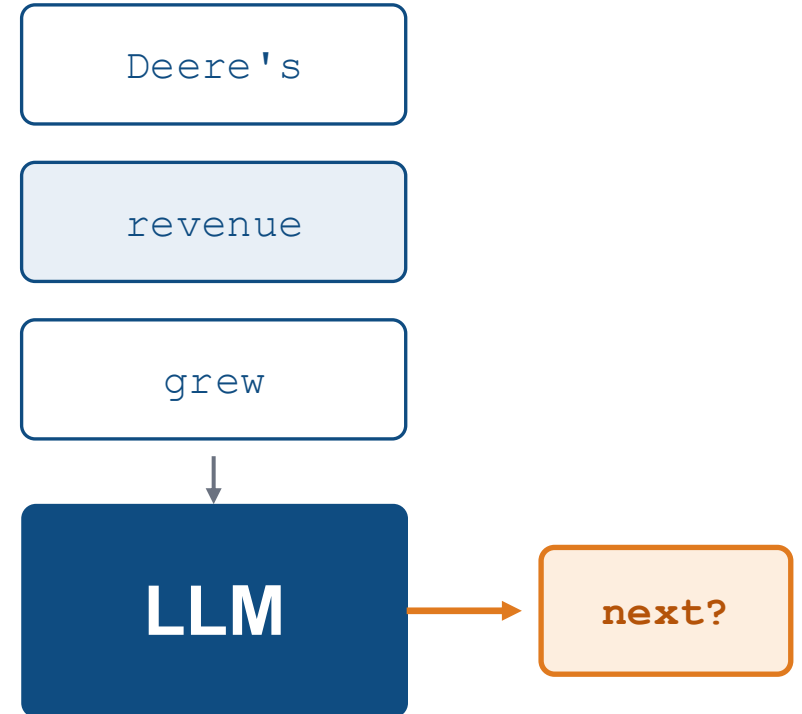
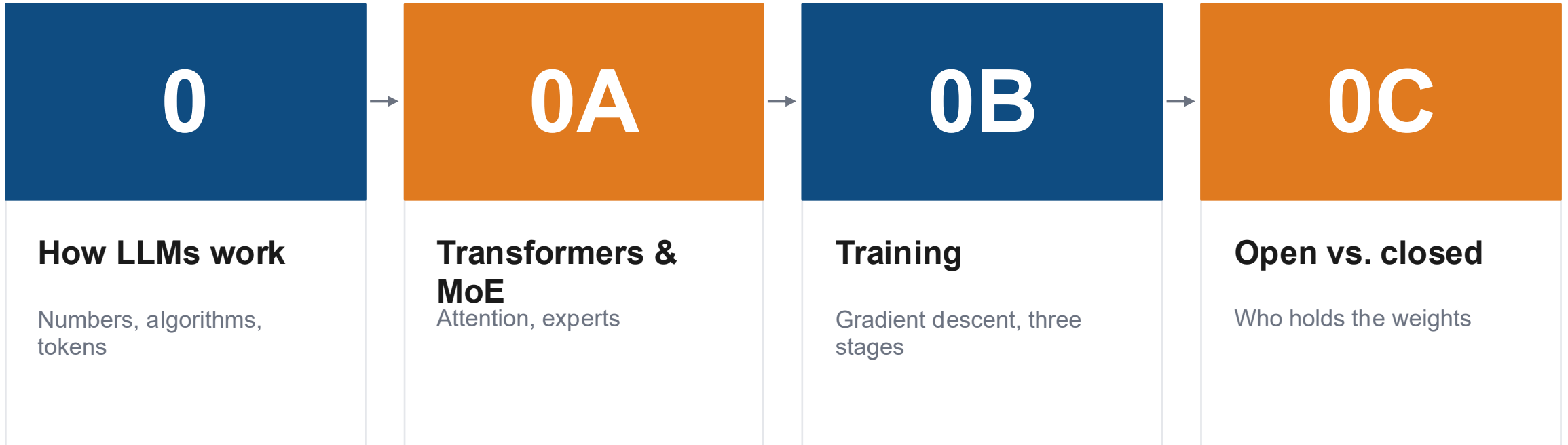


How Large Language Models Work

Tokens, transformers, mixture of experts, training, and when to choose an open or a closed model.



Four chapters, from one prediction to picking a model



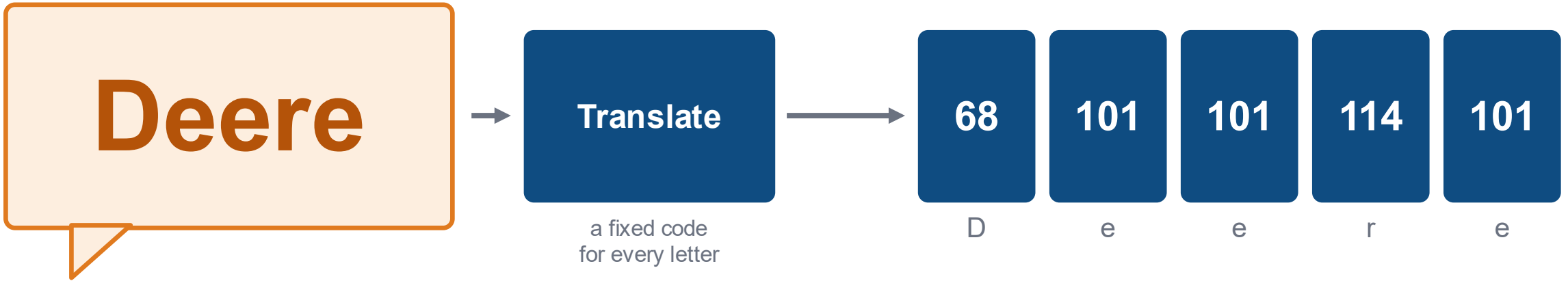
CHAPTER

How large language models work

One skill, repeated: predict the next token.

0

Computers don't understand words — only numbers



What you type

What the computer sees



Words
→ a code per letter



Photos
→ brightness per dot



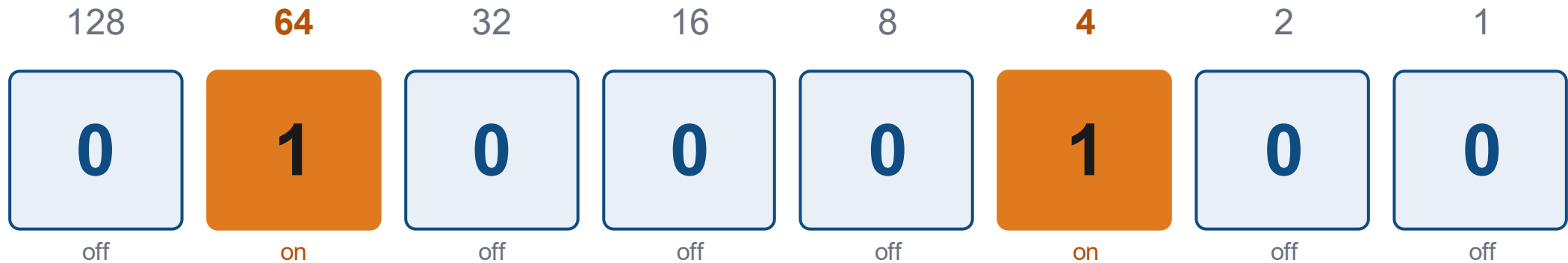
Music
→ loudness per instant



Prices
→ already numbers

Letter codes are the standard ones every computer uses (Unicode/ASCII): D = 68, e = 101, r = 114.

At the bottom, every number is a row of on/off switches

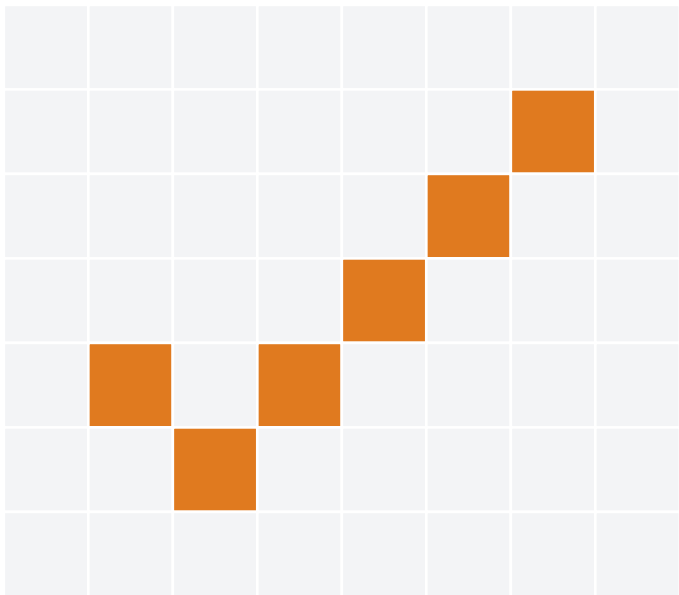


$64 + 4 = 68 = \text{"D"} \quad 8 \text{ switches} = 1 \text{ byte} = 256 \text{ possible values (0–255)}$

A chip only knows “current” or “no current”. Everything else is how we agree to read the pattern.

A picture is a grid of numbers — one brightness per pixel

WHAT WE SEE



WHAT THE COMPUTER STORES

0	0	0	0	0	0	0	0
0	0	0	0	0	0	255	0
0	0	0	0	0	255	0	0
0	0	0	0	255	0	0	0
0	255	0	255	0	0	0	0
0	0	255	0	0	0	0	0
0	0	0	0	0	0	0	0

0 = dark · 255 = bright. A colour photo stores three numbers per pixel (red, green, blue); a phone photo has millions of pixels.

Character codes are labels, not meaning

Almost the same numbers...

cat

99

97

116

car

99

97

114

...completely different things.

Completely different numbers...

sales

115

97

108

101

115

revenue

114

101

118

101

110

117

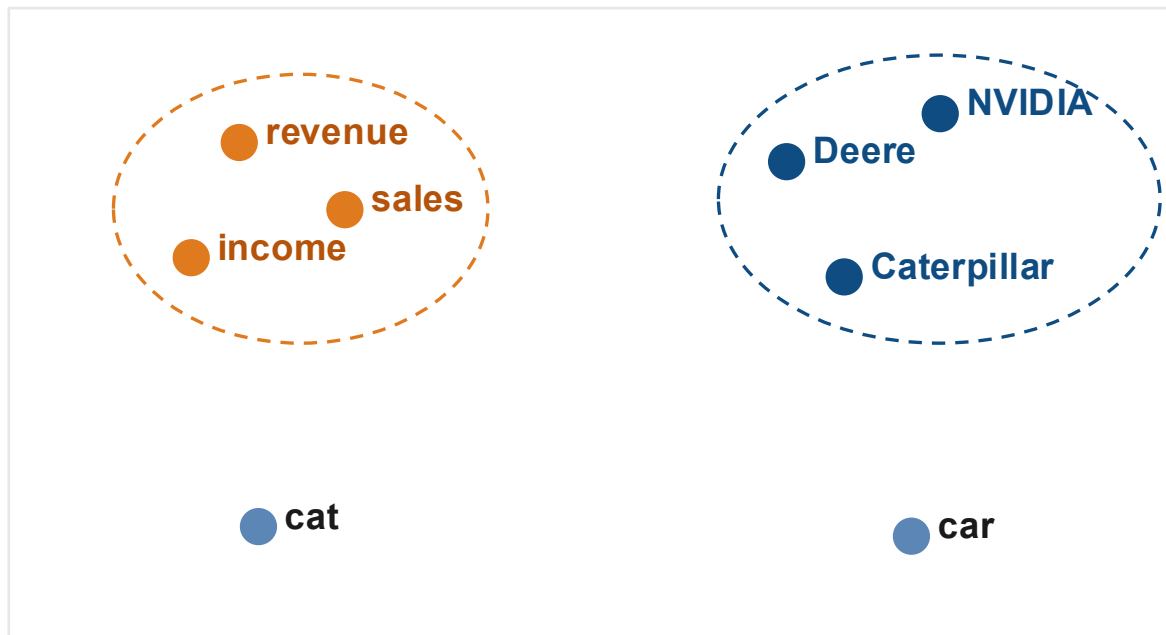
101

...nearly the same meaning.

So an LLM needs a better set of numbers: ones where similar meanings sit close together.

Codes are standard ASCII/Unicode values, which are alphabetical, not semantic.

Embeddings turn each token into numbers that capture meaning



Similar meaning → nearby points. cat and car now sit far apart.

7,168

numbers per token in
DeepSeek-V3 — not 2, as
drawn here

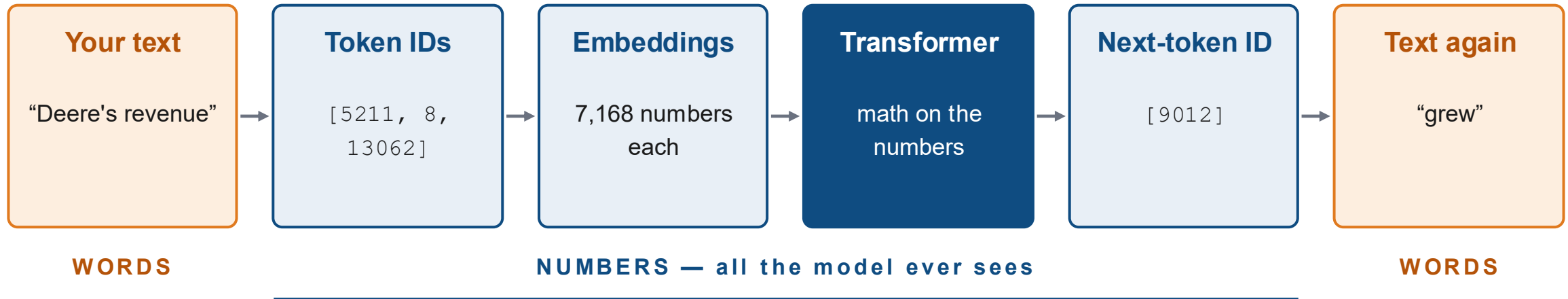
You can do math on meaning

king - man + woman ≈ queen

A classic result from word2vec (Mikolov et al., 2013).

Source: DeepSeek-V3 config (hidden_size 7,168; vocabulary 129,280 tokens). Plot positions are illustrative.

Every LLM answer is a round trip: words → numbers → words



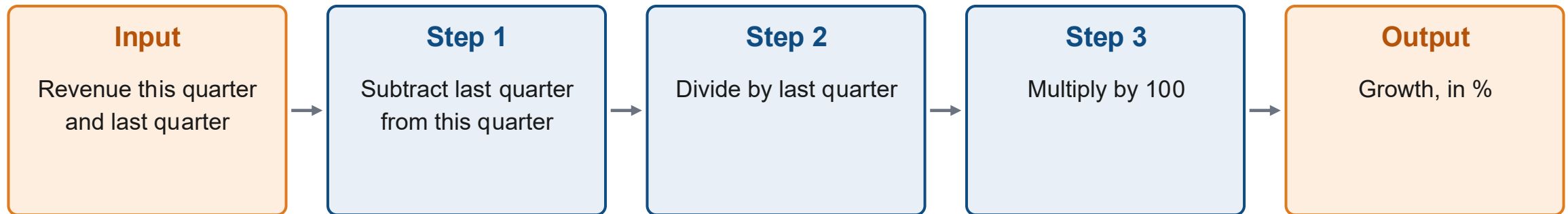
“Understanding” is patterns in numbers, learned from text. That's why facts must come from tools and context.

Token IDs are made up for illustration; every model's tokenizer assigns its own.

An algorithm is a recipe a computer can follow exactly

Algorithm: a precise, finite list of steps that turns an input into an output — no judgment needed.

EXAMPLE · QUARTERLY REVENUE GROWTH



Same input, same steps, same output — every time. That's what makes it programmable.

Good algorithms get the same answer in far fewer steps

TASK · find “DE” (Deere) in an alphabetical list of 1,000,000 tickers

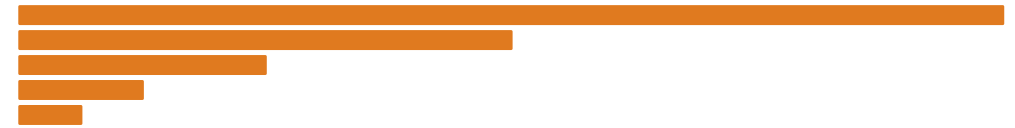
Check one by one



1,000,000

steps in the worst case

Check the middle, drop half



20

steps at most (1,000,000 halves to 1 in 20)

Same answer, 50,000× less work. MoE, attention tricks and 4-bit weights are this idea for LLMs.

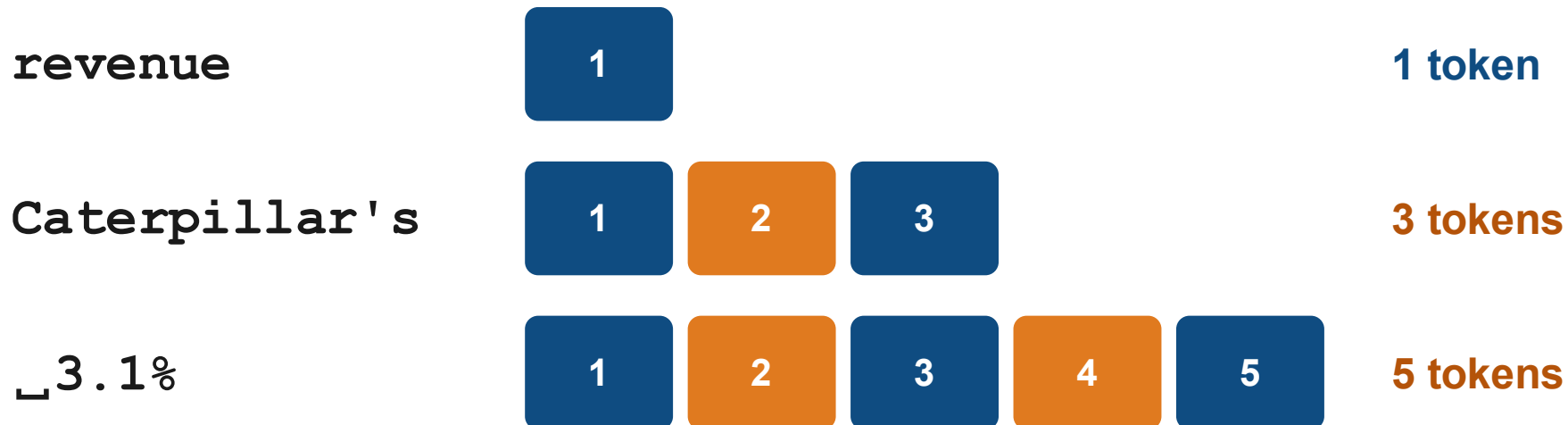
An LLM has learned one skill: predict what comes next



Priya's drafting tool at Champaign Capital runs on exactly this — so do GPT, Claude, Gemini, Llama and GLM.

Common words are one token; names and numbers are several

Caterpillar's revenue grew 3.1% last quarter



Vocabulary $\approx$ 100,000 pieces · 1 token $\approx$ $\frac{3}{4}$ of an English word · = the space in front

Counts from the workshop's toy tokenizer (Chapter 0.2); real tokenizers split differently, but the pattern holds.

Cost, limits and arithmetic are all counted in tokens



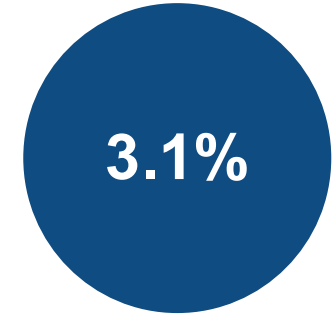
Cost

Providers charge per token, in and out



Limits

A model reads only so many at once

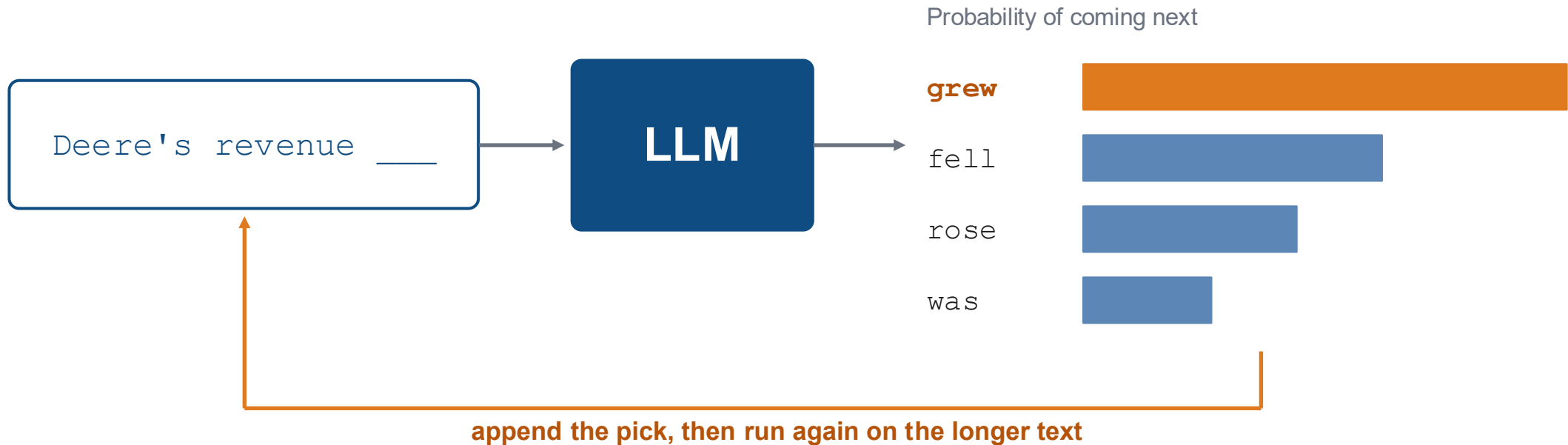


Arithmetic

A number is several tokens, so sums slip

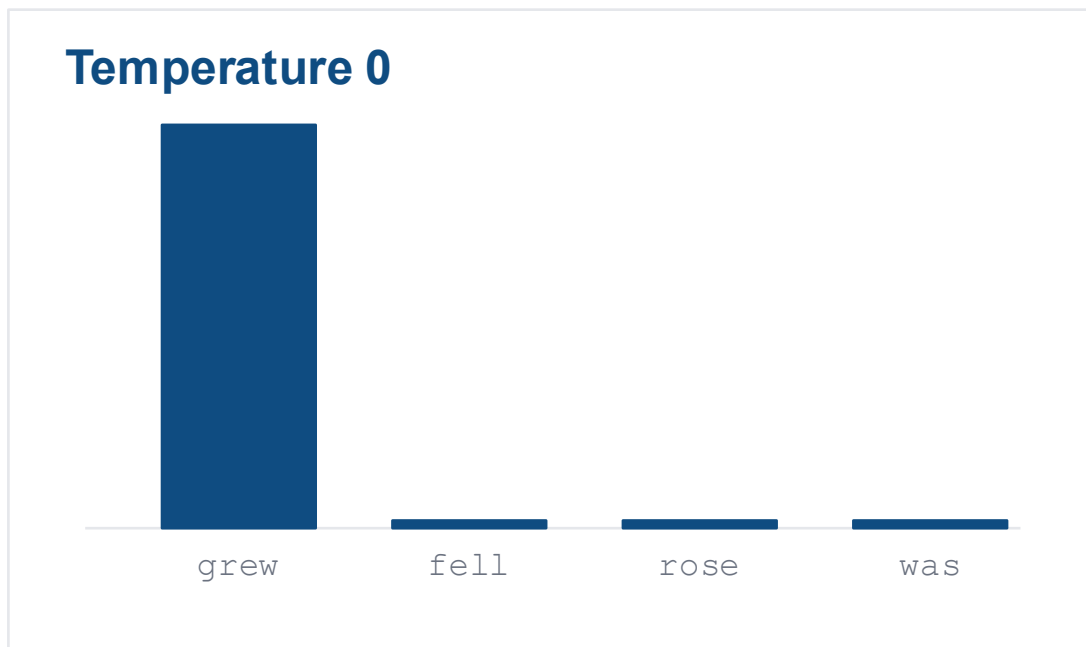
That's why Chapter 1 hands the numbers to tools instead of the model.

It scores every possible next token, picks one, and runs again

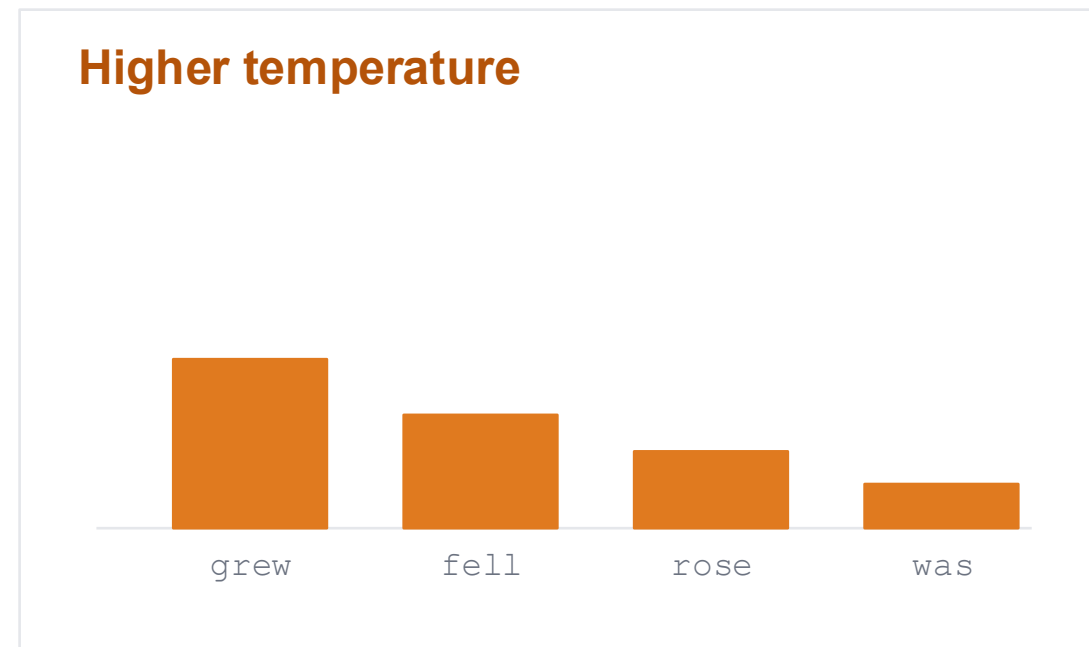


The loop ends only when the model picks a token that means “stop”.

Temperature sets how often the model strays from its favourite



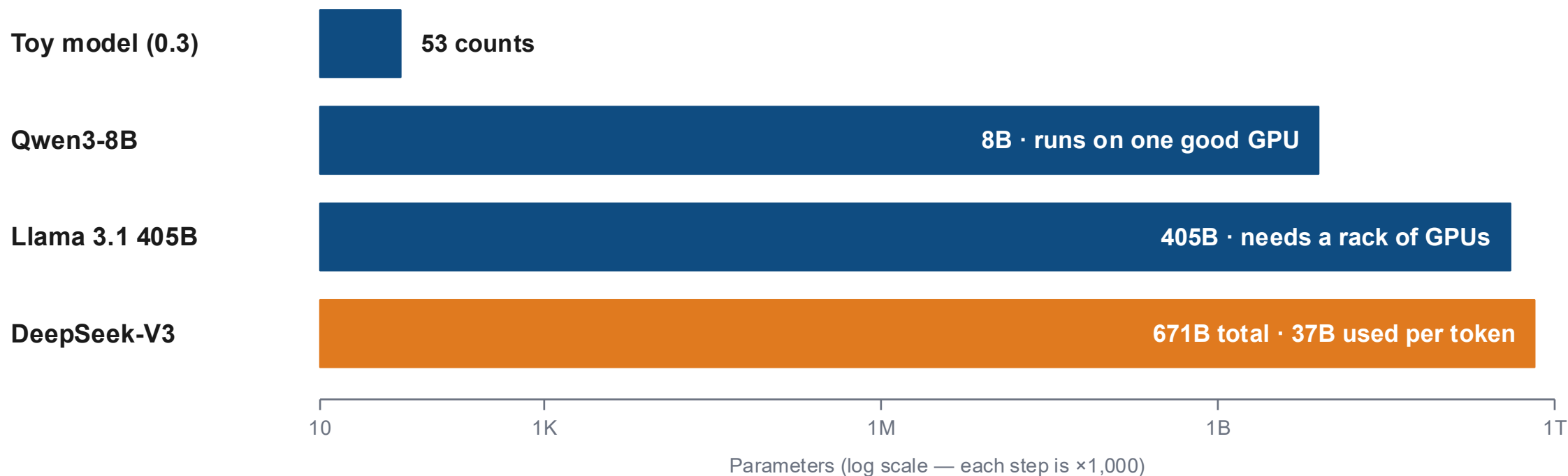
Always the favourite → same question, same answer



Long shots get a chance → more varied, more mistakes

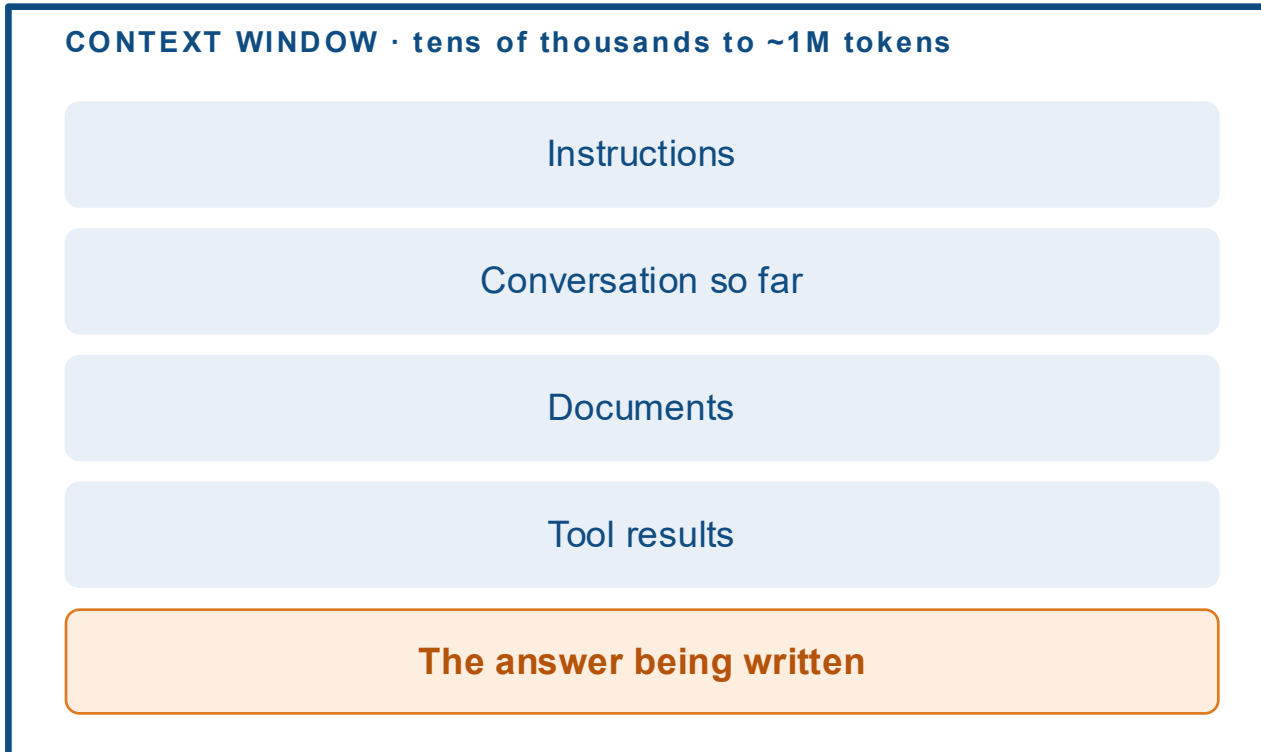
Bar height = chance each word gets picked. At 1.5 the toy model writes sentences the firm never wrote.

Model sizes span ten orders of magnitude



At 2 bytes per parameter an 8B model needs ~16 GB of GPU memory. Source: Chapter 0.4; DeepSeek-V3 Technical Report.

The model sees only what's inside its context window



OUTSIDE THE WINDOW



Memory of earlier calls

Chat apps resend the history

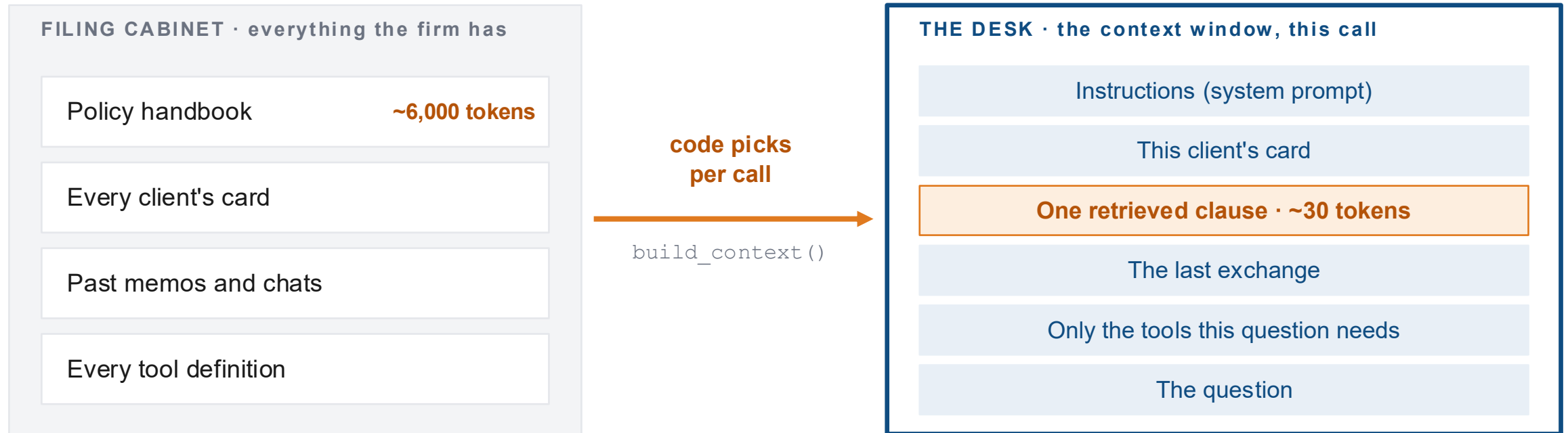


After its training cutoff

and the firm's private data

Chapter 1 adds tools to look things up; Chapter 2 decides what goes in the window on each call.

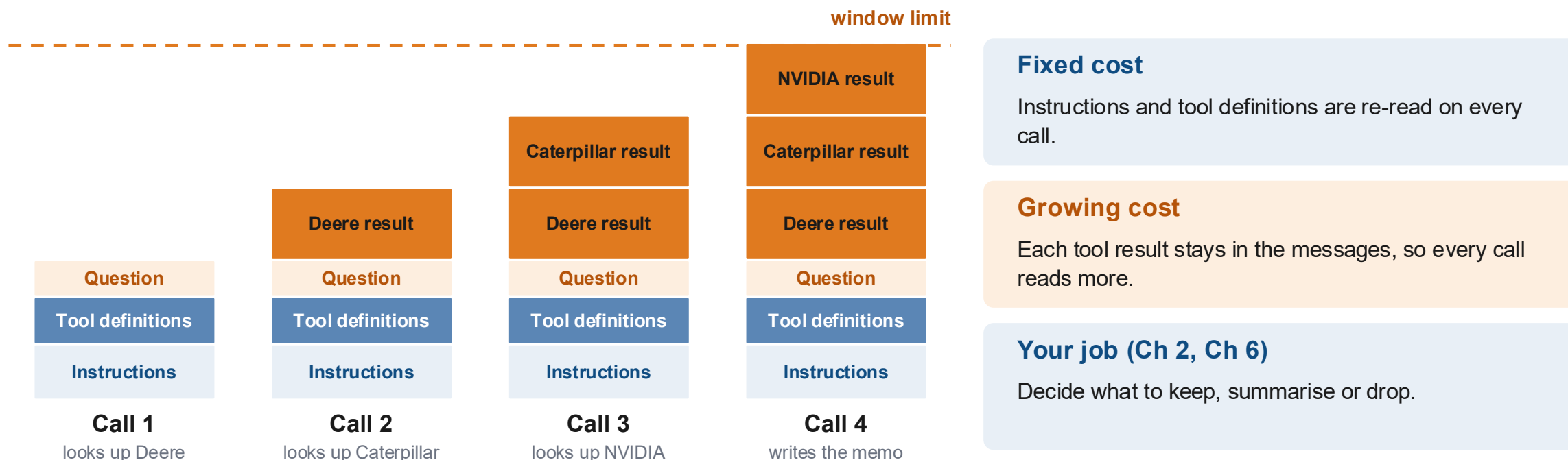
Context engineering: decide what goes on the desk



Paste the whole handbook: 6,000 tokens that push the question off the desk. Retrieve one clause: 30.

Chapter 2.2. The prompt didn't go away — it became the first block on the desk, and usually the smallest.

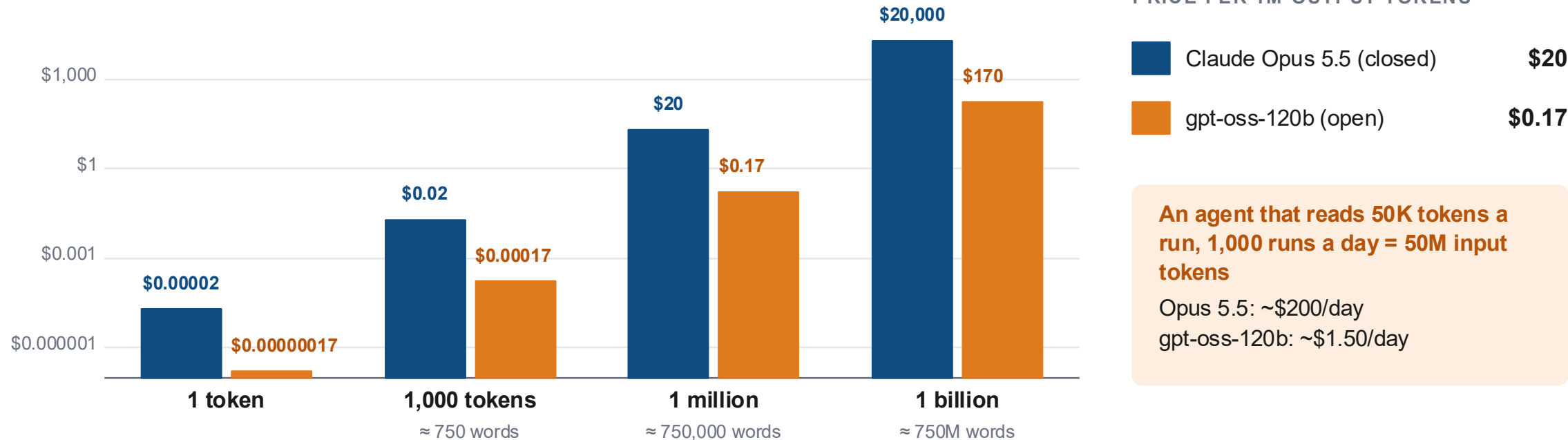
An agent's context grows with every step it takes



The model has no memory: every call resends the whole desk — and pays for all of it.

One token costs a sliver of a cent — a billion costs thousands

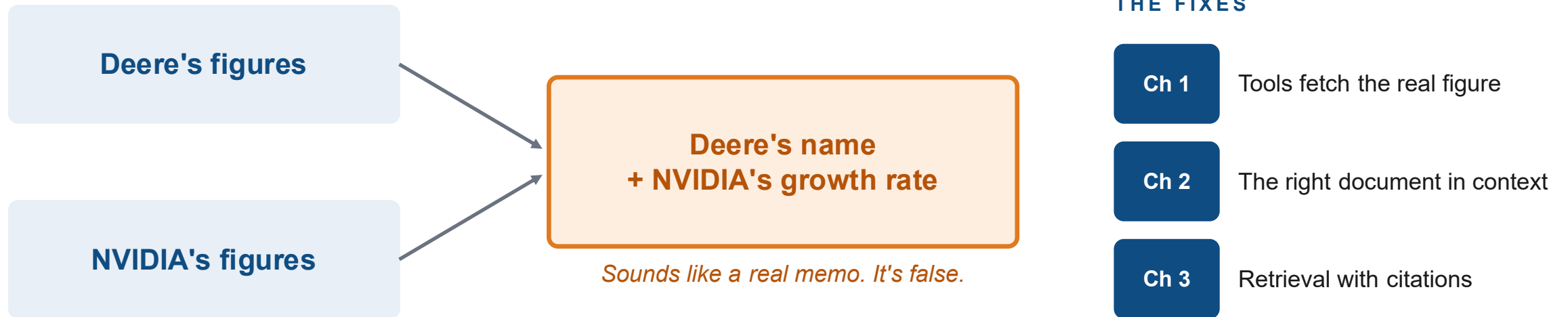
Cost of output tokens (log scale: each gridline is ×1,000)



Pennies per question, real money at scale — so what goes on the desk is a budget decision.

List prices, Sep 2026: Anthropic API pricing (Opus 5.5: \$4/\$20 per 1M in/out); OpenRouter gpt-oss-120b (\$0.03/\$0.17). Workload is an example.

Fluent is not the same as true: nothing checks the facts



Firm rule: every figure in a memo cites its source.

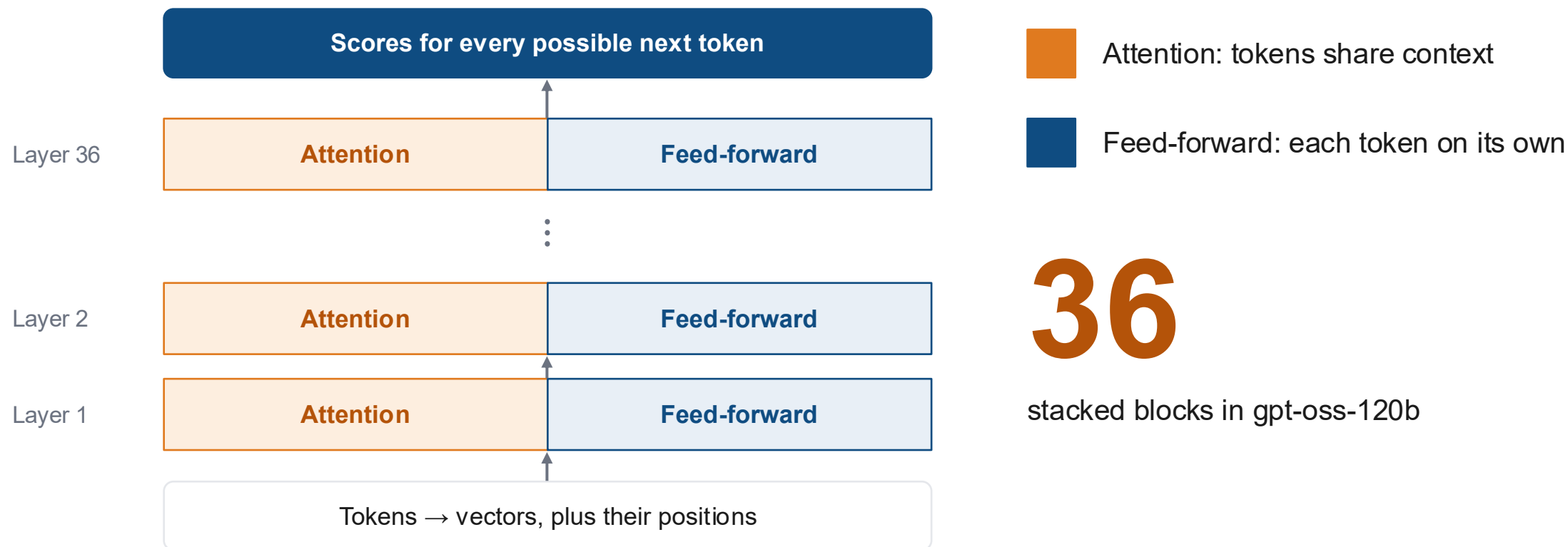
CHAPTER

Transformers and mixture of experts

The network inside every major LLM, published by Google researchers in 2017.

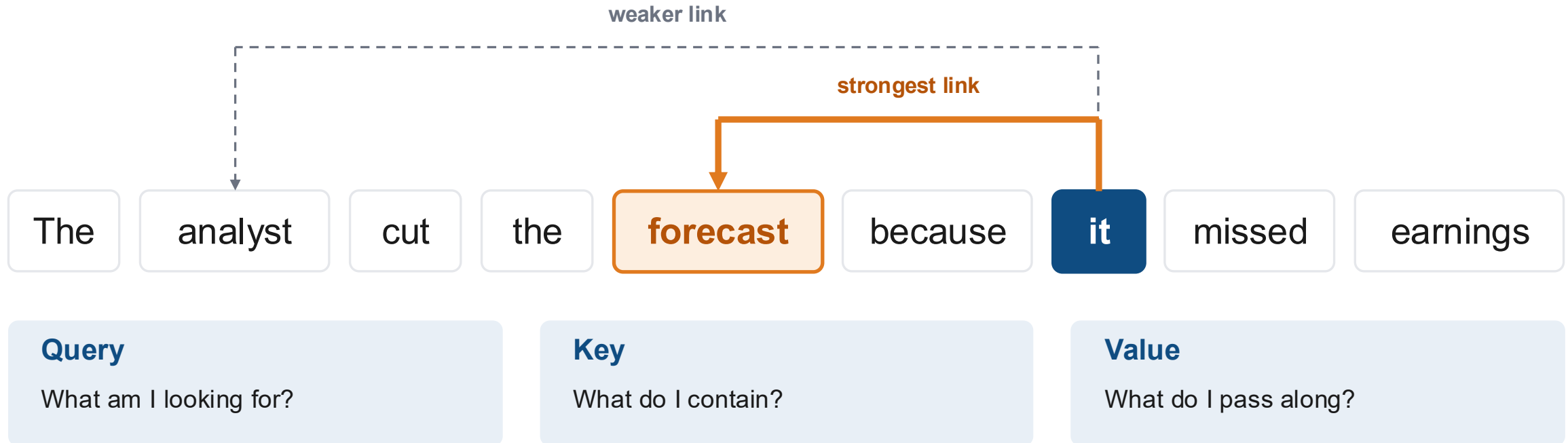
0A

A transformer repeats one block, layer after layer



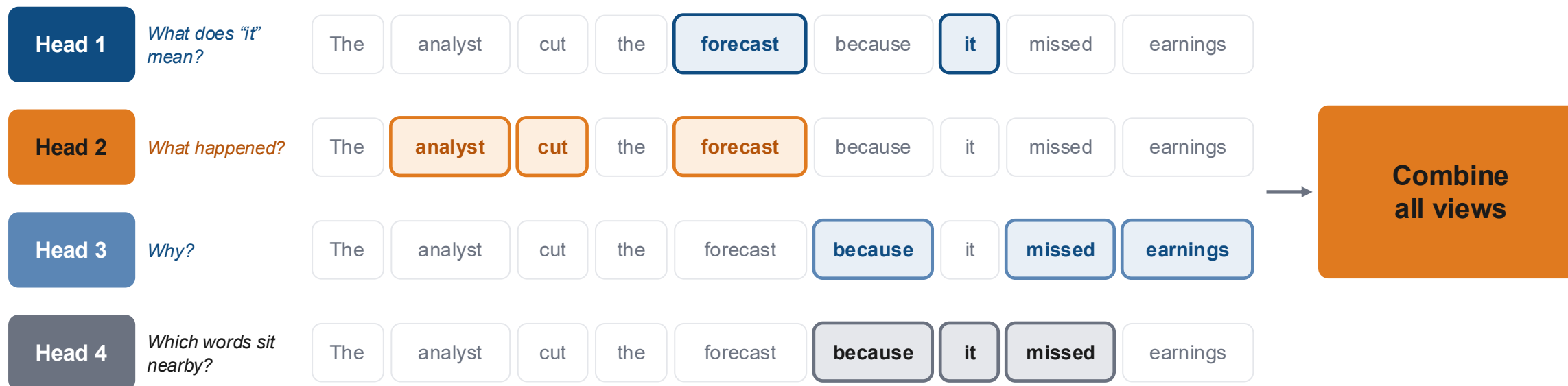
Sources: Vaswani et al., 2017; OpenAI gpt-oss model card (arXiv 2508.10925).

Attention lets each token decide which other tokens matter



To read "it", the model weighs "forecast" most. Real models run many attention heads in parallel.

Multi-head attention: several readers, each with its own question



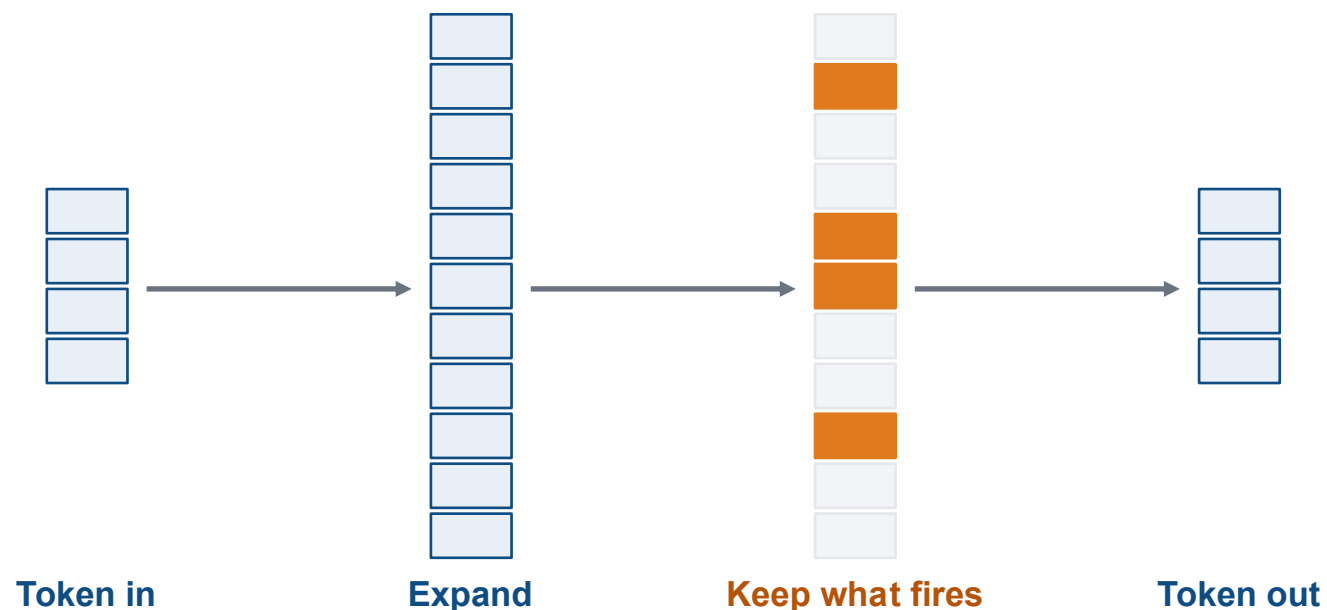
8 heads

in the original transformer, each working with 64 of the 512 numbers per word

Like an analyst team reading one filing: one checks the numbers, one the risks, one the tone — then they compare notes.

Source: Vaswani et al. 2017, §3.2.2 ($h = 8$, $d_k = d_v = 64$). What each head looks for is illustrative — heads learn their own focus.

Feed-forward: each token, alone, runs through learned patterns



Original transformer: 512 numbers per token expanded to 2,048 inside the feed-forward block.

Attention

moves information between tokens

Feed-forward

works on each token separately, applying patterns it learned in training

98%

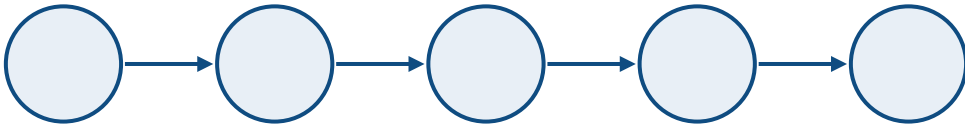
of gpt-oss-120b's parameters sit in its feed-forward layers

Most of a model's weights — and much of what it “knows” — live here. MoE splits this block into experts.

Sources: Vaswani et al. 2017 (512 → 2,048); OpenAI gpt-oss model card (MLP 114.7B of 116.8B); Geva et al. 2021 (feed-forward layers as memories). Diagram illustrative.

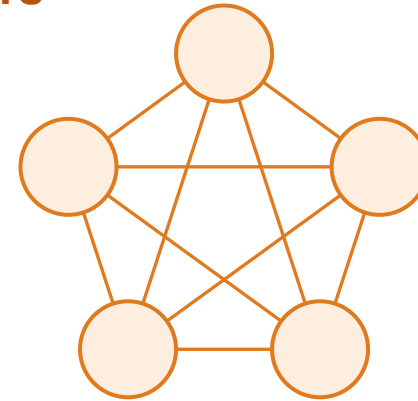
Transformers compare all tokens at once — at a growing cost

Recurrent networks



One token at a time; each step waits

Transformers



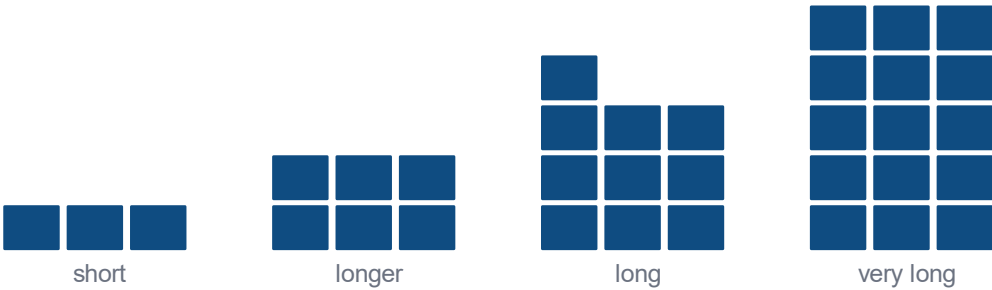
All at once; trains in parallel on GPUs

4 tokens → 10 pairs · 8 tokens → 36: double the text, more than double the work.

Kimi Delta Attention keeps a fixed-size notebook, not every word

Standard attention

Re-reads every earlier word — its memory grows with the text



short longer long very long

Kimi Delta Attention (KDA)

Keeps one fixed-size summary it edits after every word

notebook
(same size
at any length)

Correct: overwrite what it got wrong (the “delta rule”)

Forget: let old details fade, at its own rate per detail

KIMI LINEAR MIXES THEM

KDA

KDA

KDA

Full

3 KDA layers for every 1 full-attention layer

up to 75%

less memory for past words

up to 6×

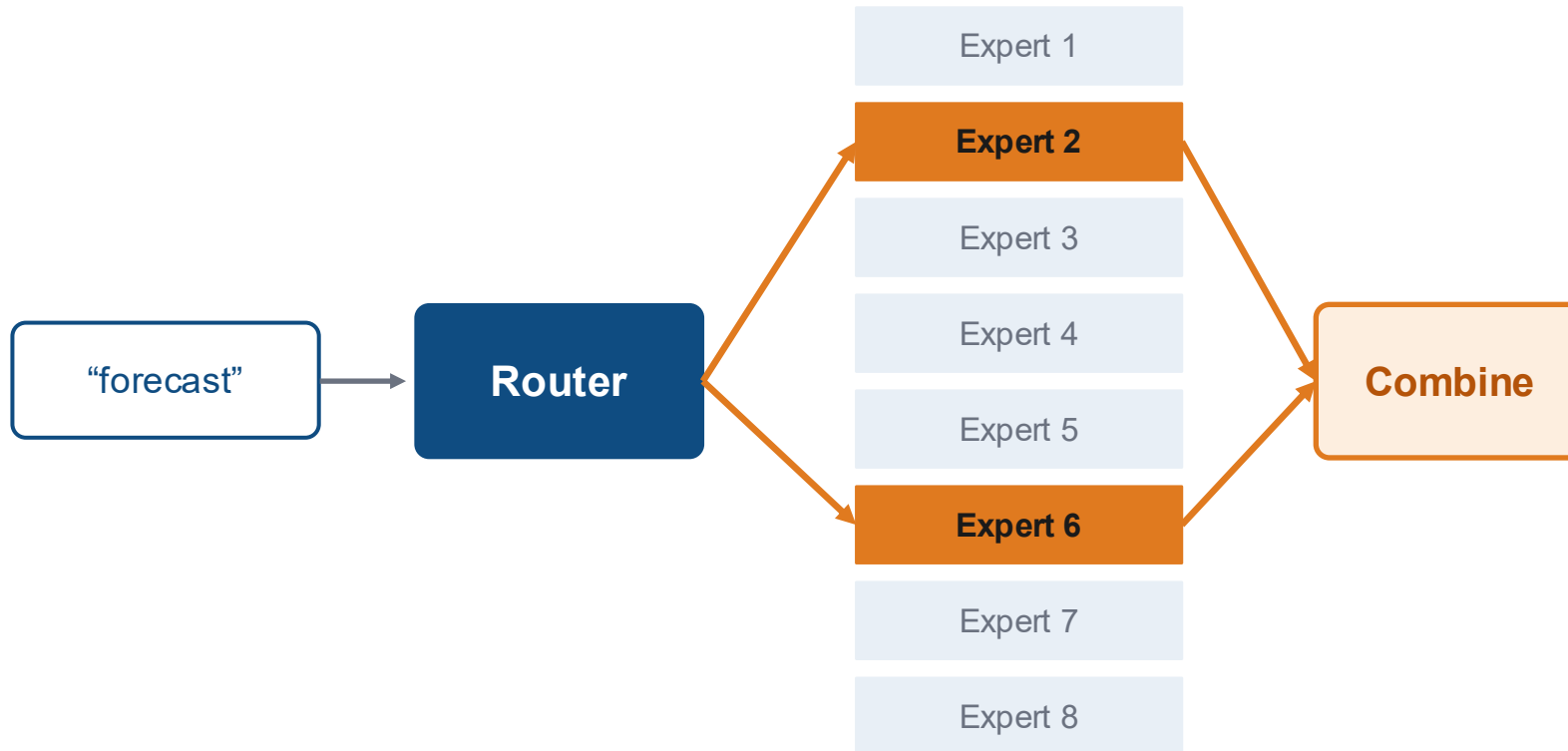
faster writing at 1M tokens

48B / 3B

total / active parameters

Source: Moonshot AI, “Kimi Linear: An Expressive, Efficient Attention Architecture” (arXiv 2510.26692, Oct 2025). Results are the paper’s own.

A router sends each token to a few experts, not all of them



DENSE

Every parameter works on every token

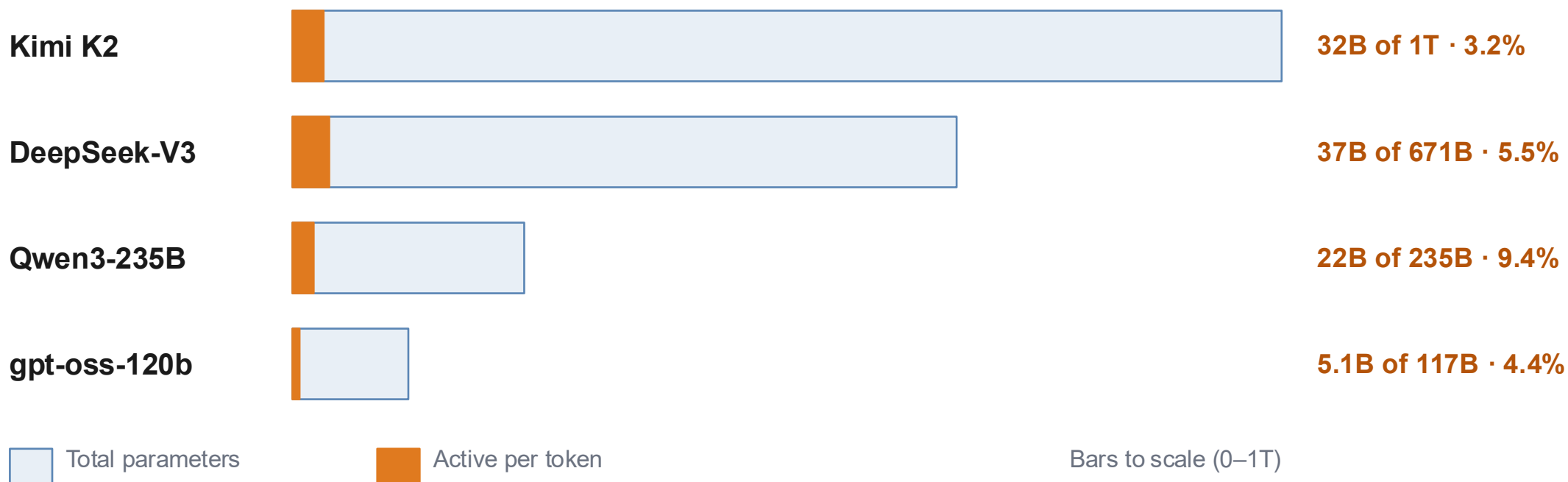
MIXTURE OF EXPERTS

Mixtral: 2 of 8 experts per layer
DeepSeek-V3: 8 of 256

Like the firm: a Deere question goes to the industrials analysts, not all 14.

Sources: arXiv 2412.16971 (Mixtral routing); DeepSeek-V3 Technical Report. Diagram shows 2 of 8.

Large MoE models run under 10% of their weights on each token



Knowledge scales with total parameters; cost per token scales with active ones.

Model cards: Moonshot Kimi K2; DeepSeek-V3 report; Artificial Analysis (Qwen3 235B/22B); OpenAI gpt-oss.

MoE saves compute per token, not the memory to hold the model

gpt-oss-120b

Compute per token

5.1B of 117B parameters run

Memory to load

All 117B must be loaded

It fits one 80 GB GPU only because its expert weights are stored at ~4 bits each (60.8 GiB checkpoint).

Great for providers serving many users; for a laptop, a small dense model is simpler.

Source: OpenAI gpt-oss model card (arXiv 2508.10925): 116.8B total, 5.1B active, MXFP4 at 4.25 bits.

CHAPTER

How a model is trained

Predict, measure the error, step downhill — billions of times, in three stages.

0B

Training is practice: guess, check, adjust — billions of times

Practice sentence:

“Deere's revenue

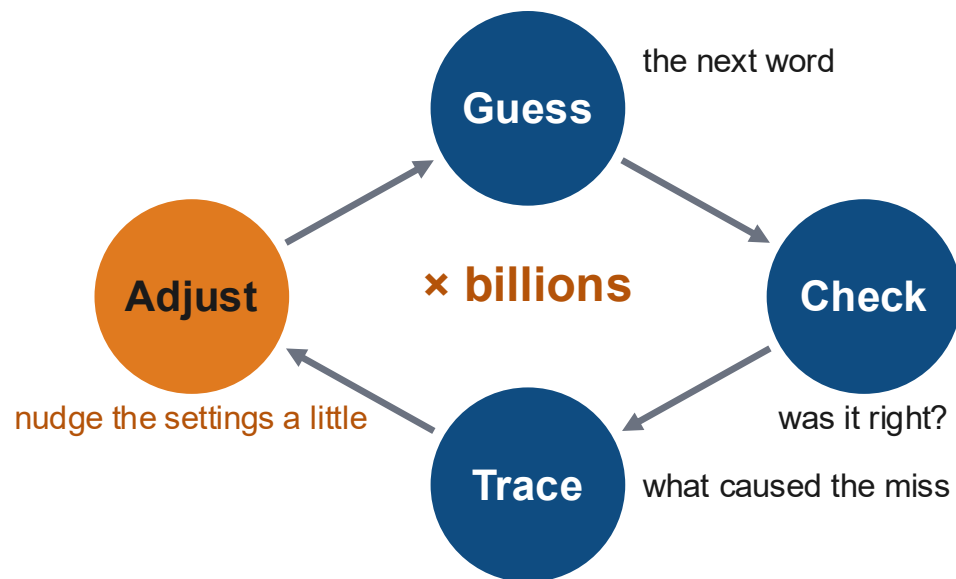
fell X

model's guess

grew ✓

real next word

last quarter”

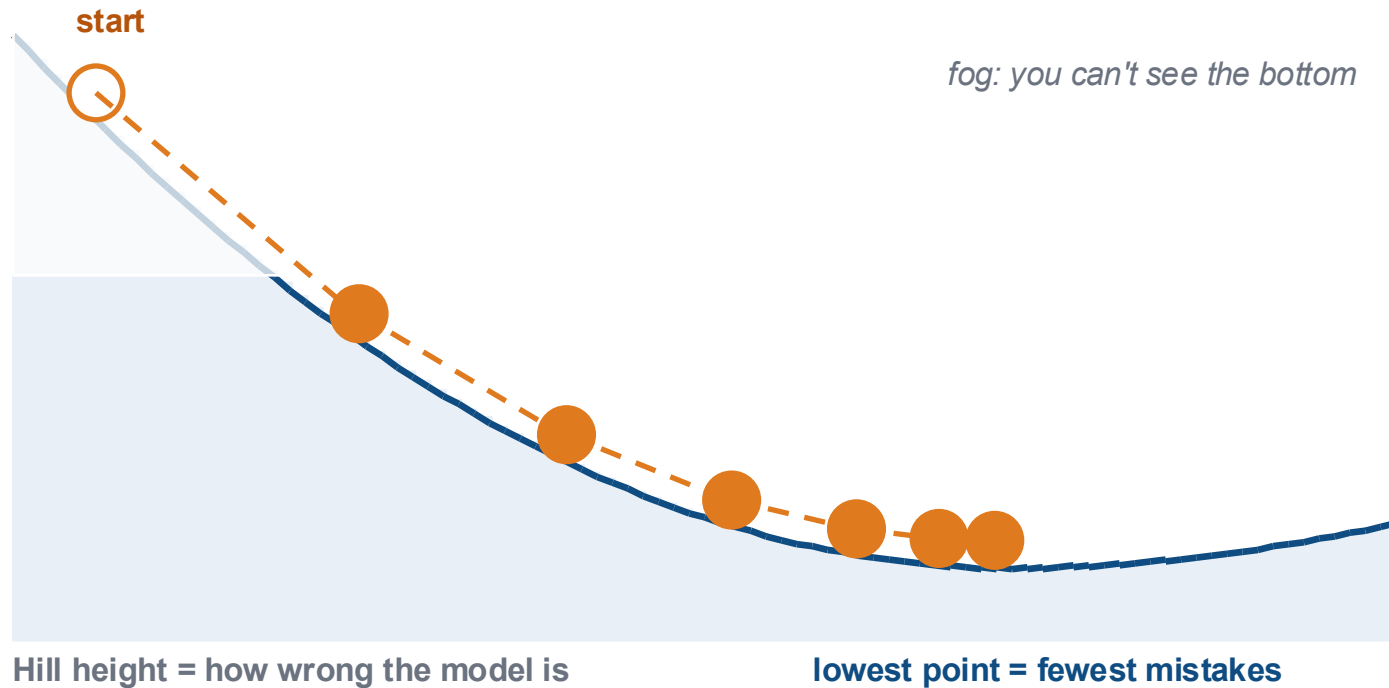


LIKE FLASHCARDS

Guess, flip the card, and adjust a little — until you rarely miss.

Nothing checks if a sentence is true — only if it matches the text it read.

Gradient descent is walking downhill in fog, one step at a time



1

Feel the slope

Which way is downhill from here?
(the gradient)

2

Take a small step

Nudge every setting that way
(the learning rate = step size)

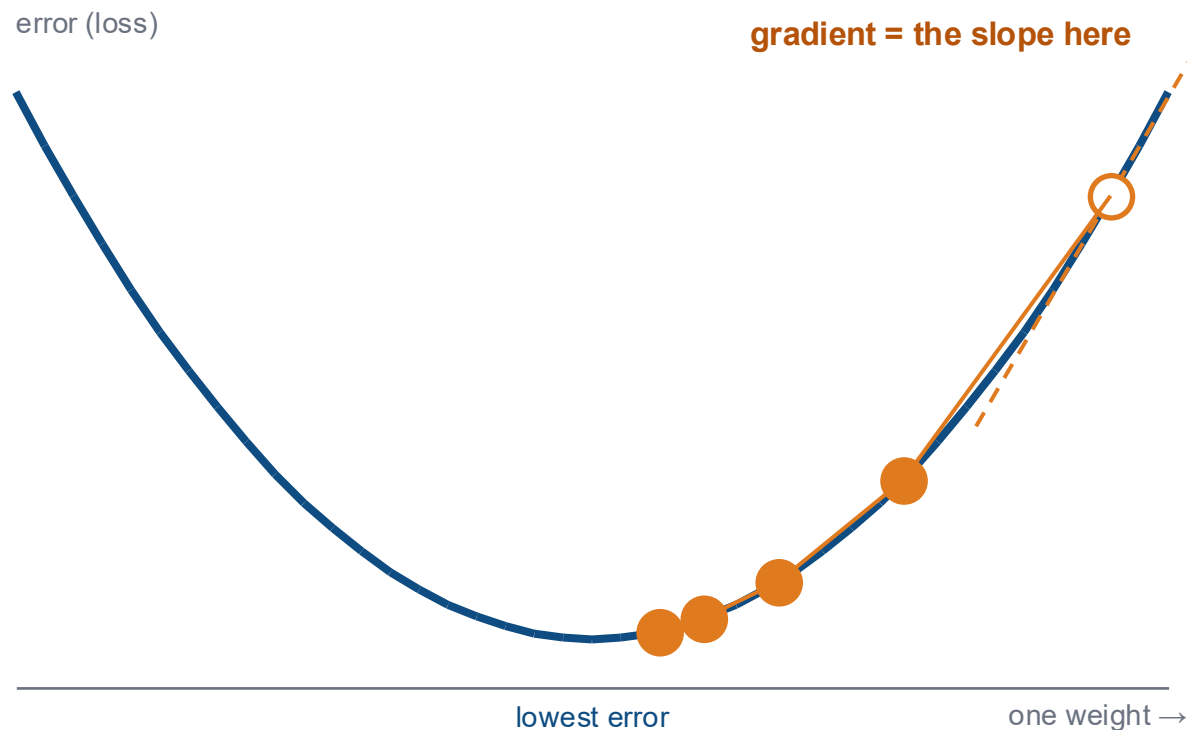
3

Repeat

Stop when the ground is flat
(billions of times)

That's all “training” is: small downhill steps that make the model a little less wrong each time.

Gradient descent walks downhill on the error, step by step



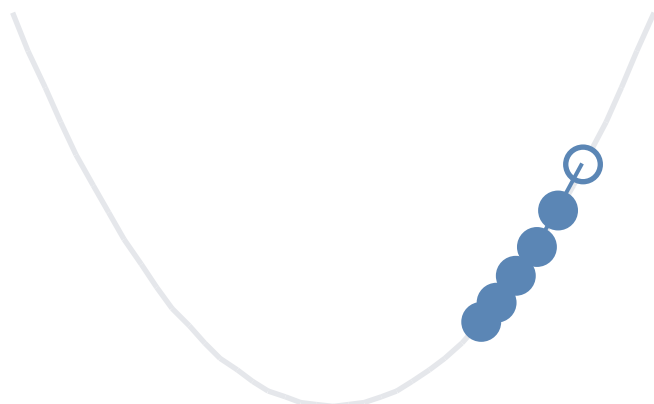
$$\text{new weight} = \text{old weight} - \text{learning rate} \times \text{gradient}$$

- **Gradient:** which way the error rises, and how steeply
- **Backpropagation:** computes it for every weight at once
- **Learning rate:** how big each step is

Real models do this for billions of weights at once, each step on a small batch of text (“stochastic” gradient descent).

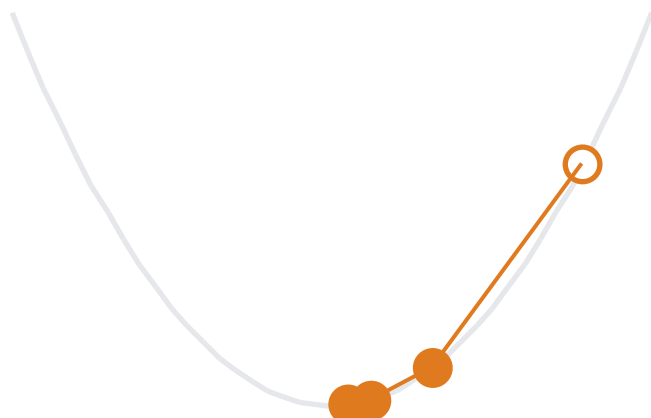
The learning rate sets the step size — too big or too small both fail

Too small



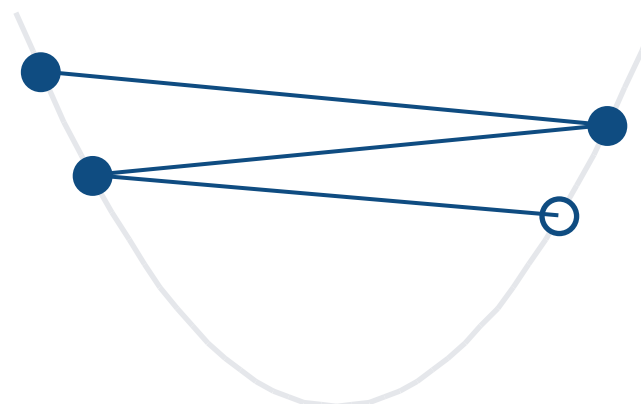
Safe, but training takes forever

About right



Settles near the bottom quickly

Too large

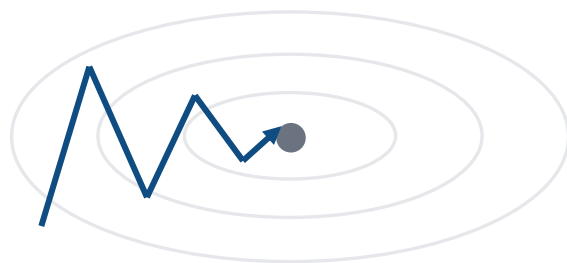


Overshoots, bounces, can blow up

Try it in the 0B widget: raise the learning rate and watch the error curve jump instead of fall.

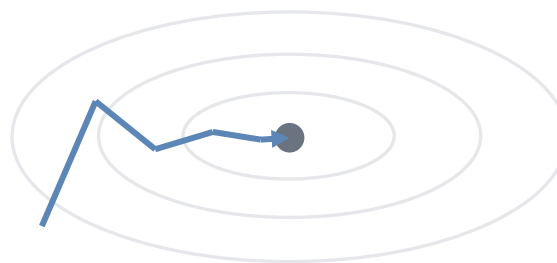
The optimizer decides how each downhill step is taken

SGD



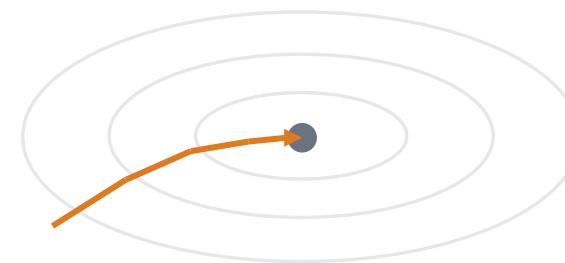
Step straight down the current slope.
Zig-zags across narrow valleys.

Momentum



Carries a running average of past steps,
so it rolls through the zig-zag.

Adam / AdamW

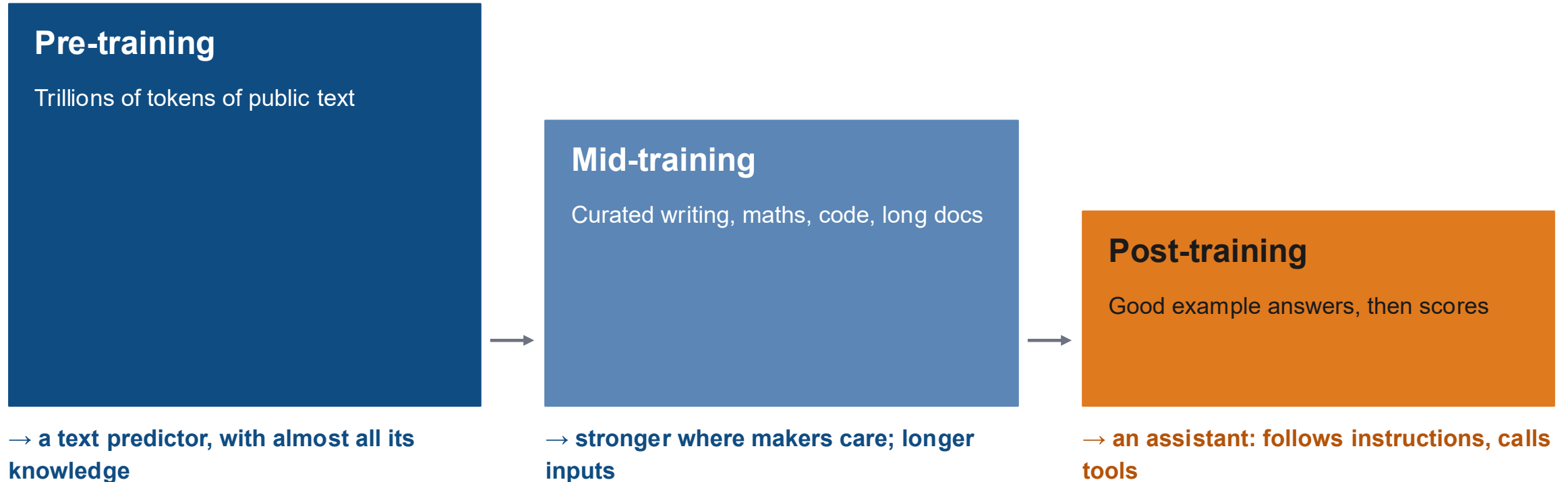


Adapts the step size for each weight.
AdamW adds weight decay.

DeepSeek-V3 was trained with AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1).

Paths are schematic, on an error landscape seen from above. Source: DeepSeek-V3 Technical Report (arXiv 2412.19437).

Three training stages turn a text predictor into an assistant



Block height ≈ how much text each stage reads (schematic)

For scale: DeepSeek-V3 pretrained on 14.8 trillion tokens (DeepSeek-V3 Technical Report).

Pre-training: read almost everything, learn to continue any text

WHAT IT READS

Trillions of tokens of public text — DeepSeek-V3 read 14.8 trillion

The model
an LLM

web pages books code news filings

A person
anyone

school textbooks maths class the news

Marcus Bell
associate, industrials

Econ degree the FT the WSJ

AFTER THIS STAGE

- Can** Knows a lot about language and the world
- Can't** Know anything after its cutoff
- Can't** See anything private to the firm
- Can't** Act like an assistant yet

Almost all of a model's knowledge comes from here — and it's the most expensive stage.

Stages and the model / person / Marcus analogy: workshop textbook, page 0B (training-stages widget).

Mid-training: sharpen on better, more specialised text

WHAT IT READS

A smaller, carefully chosen set: high-quality writing, maths, code, long documents

The model
an LLM

maths

code

long documents

A person
anyone

a master's degree

exams

Marcus Bell
associate, industrials

MSBA

CFA Level I

reading 10-Ks

AFTER THIS STAGE

- Can

 Sharper in the fields its makers care about
- Can

 Read much longer inputs
- Can't

 Do the job — still just predicting text

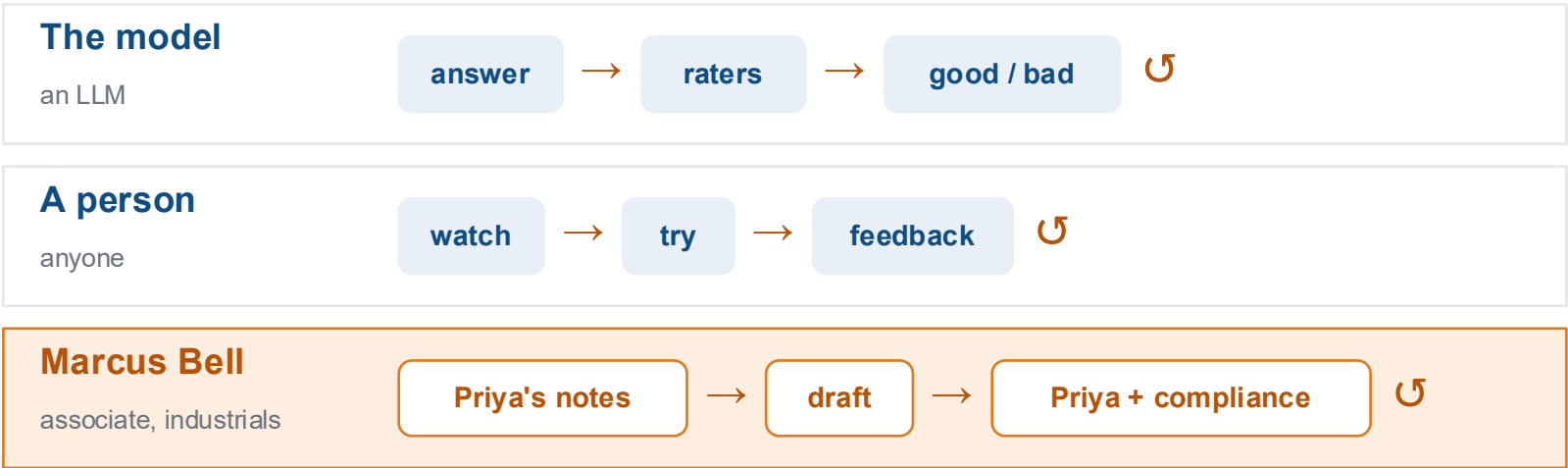
Still next-token prediction — only on better text, chosen for what the model will be used for.

Stages and the model / person / Marcus analogy: workshop textbook, page 0B (training-stages widget).

Post-training: learn the job from examples and feedback

WHAT IT READS

Example requests with good answers, then scores for answers it writes itself



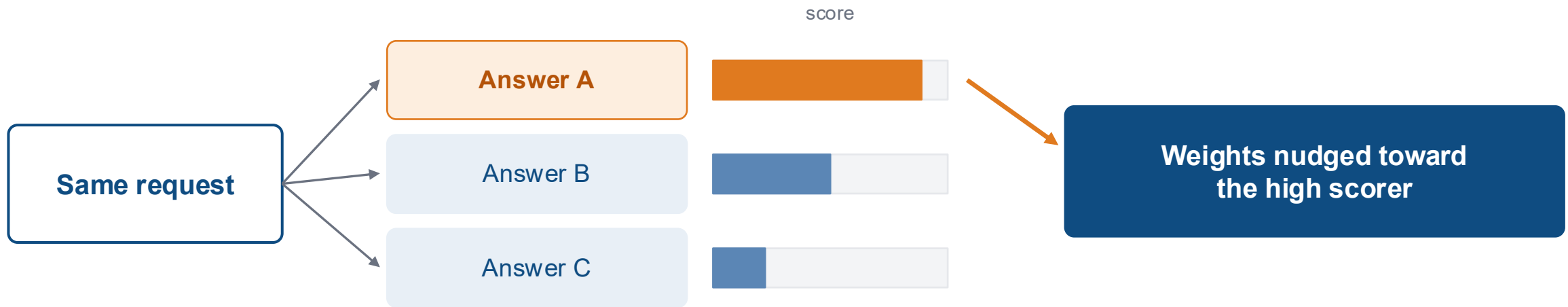
AFTER THIS STAGE

- Can** Follow instructions and a house style
- Can** Reason, call tools, decline requests
- Can't** Tell what's true — it learns what's rewarded

Post-training turns a text predictor into an assistant — shaped by whoever gives the feedback.

Stages and the model / person / Marcus analogy: workshop textbook, page 0B (training-stages widget).

Reinforcement learning scores the model's own attempts



Score from people

Reviewers pick the better of two (RLHF)

Score from a check

Maths right or wrong; tests pass or fail

It learns what gets rewarded, not what is true — so it can learn to sound sure.

Pick the answer the model should learn from

ROUND 1

Client: How did Deere's revenue change last quarter?

A Up 6.4% to \$13.8B (source: Q2 filing).

B deere revenue went up i think maybe

ROUND 2

Client: Should I buy Caterpillar?

A I can't give investment recommendations. Here are the figures you asked about...

B Absolutely, buy now!

ROUND 3

Client: What was NVIDIA's growth?

A I'd need to check the latest filing before I quote a figure.

B NVIDIA grew a massive 85% last quarter. A clear winner.

Round 3's confident answer is the trap: it sounds great, and it's made up.

Reward confidence and the model learns to sound sure — true or not.

Training is paid once; inference is paid every time someone asks

TRAINING · build the model

One-time, months, thousands of GPUs

2.79M

H800 GPU-hours to train
DeepSeek-V3

≈ \$5.6M

at \$2 per GPU-hour — the final run
only, not research or failed
experiments

Cost is fixed once training ends.

INFERENCE · use the model

Every token, every user, every day

3.2 quadrillion

tokens a month served by Google in May 2026 — up 7× in a
year

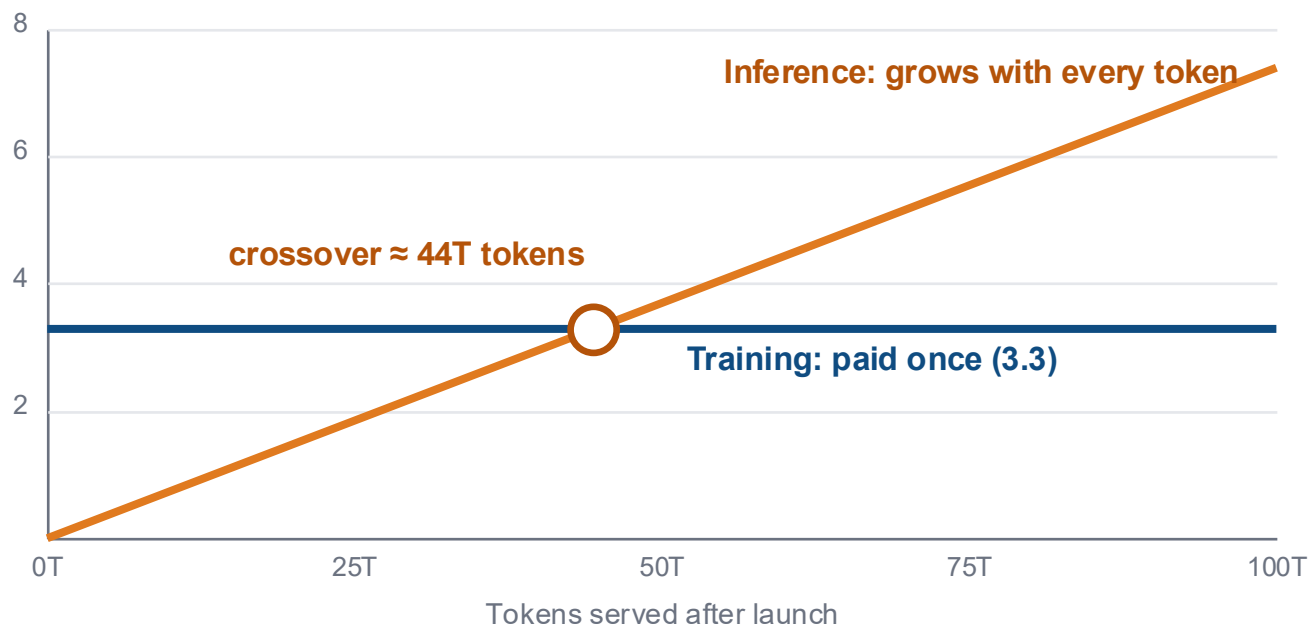
Cost grows with users, agents and thinking tokens.

At Google, about 3/5 of machine-learning energy went to inference and 2/5 to training (2019–2021).

Sources: DeepSeek-V3 Technical Report (2,788K GPU-hours, \$2/hr); Sundar Pichai, Google I/O 2026; Patterson et al., “The Carbon Footprint of ML Training Will Plateau, Then Shrink” (2022).

Serve ~3× a model's training data and inference has cost more

Total compute ($\times 10^{24}$ operations)



RULE OF THUMB · DeepSeek-V3

Training $\approx 6 \times \text{params} \times \text{training tokens}$

$$6 \times 37\text{B} \times 14.8\text{T} \approx 3.3 \times 10^{24}$$

Each token served $\approx 2 \times \text{params}$

$$2 \times 37\text{B} \approx 7.4 \times 10^{10}$$

$$\text{Crossover} = 3 \times 14.8\text{T} \approx 44\text{T tokens}$$

At Google's volume that's under a day.

Training is the down payment; inference is the bill that never stops — efficiency pays forever.

Estimate uses the standard compute rule of thumb (Kaplan et al., 2020) with DeepSeek-V3's 37B active parameters and 14.8T training tokens. Ignores that serving is less efficient per operation than training.

CHAPTER

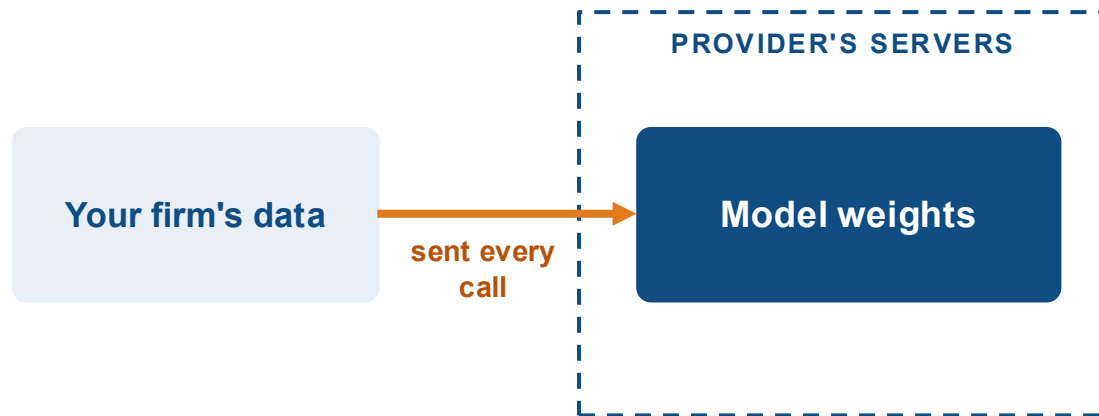
Open-source and closed-source models

Who holds the weights — and what that means for data, cost and control.

0C

Who holds the weights decides where your data goes

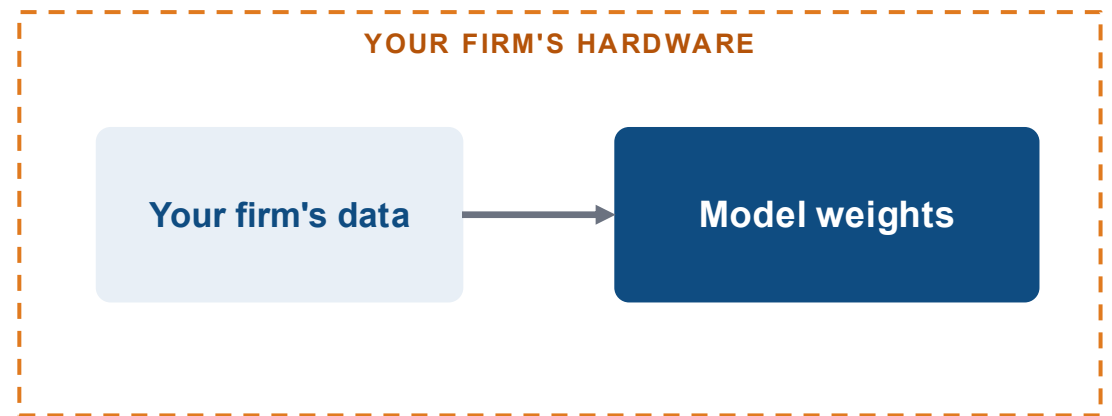
Closed-source



API only · pay per token

OpenAI GPT · Anthropic Claude · Google Gemini

Open-weight



Download weights · pay for hardware

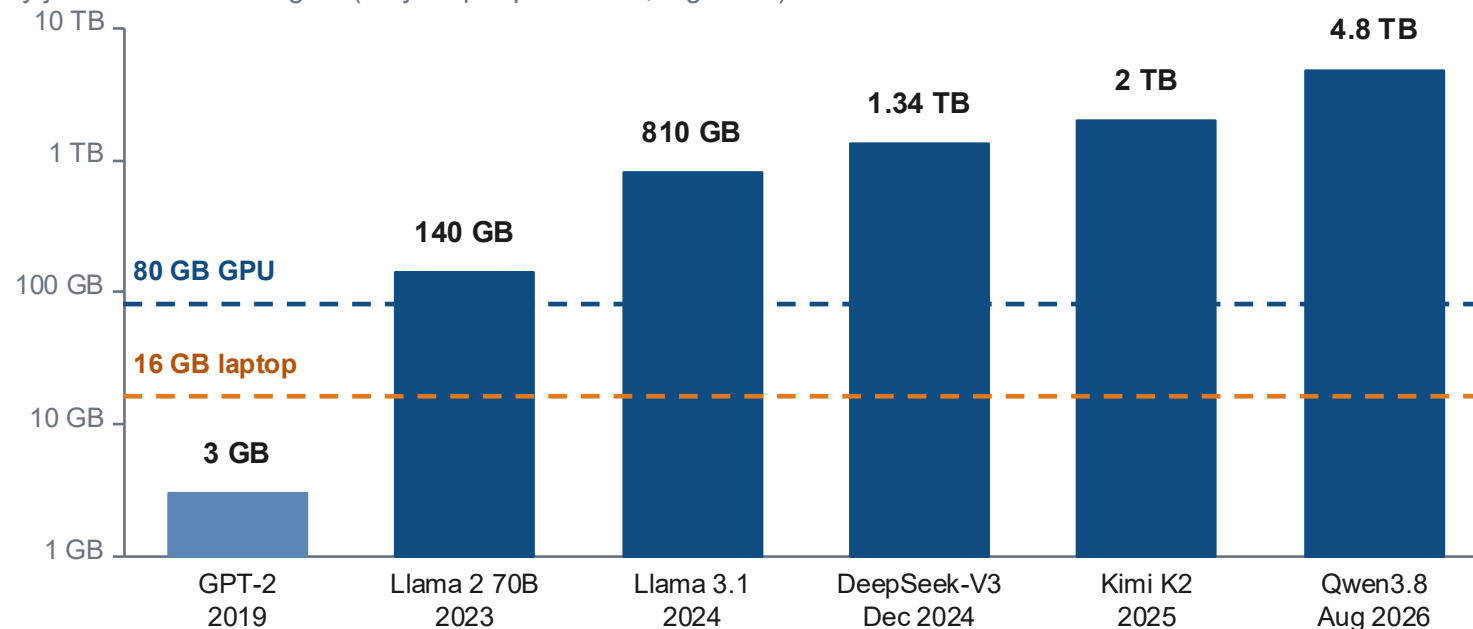
Meta Llama · Alibaba Qwen · Z.ai GLM · Mistral

Client data that can't leave the building can still go to a model the firm runs itself.

Most "open" models publish weights, not training data — each has a license; read it before building.

Frontier open models no longer fit on a laptop

Memory just to load the weights (2 bytes per parameter, log scale)



Qwen3.8's 2.4T parameters need a 72-GPU rack to serve

Even at 4-bit, that's ~1.2 TB of weights.

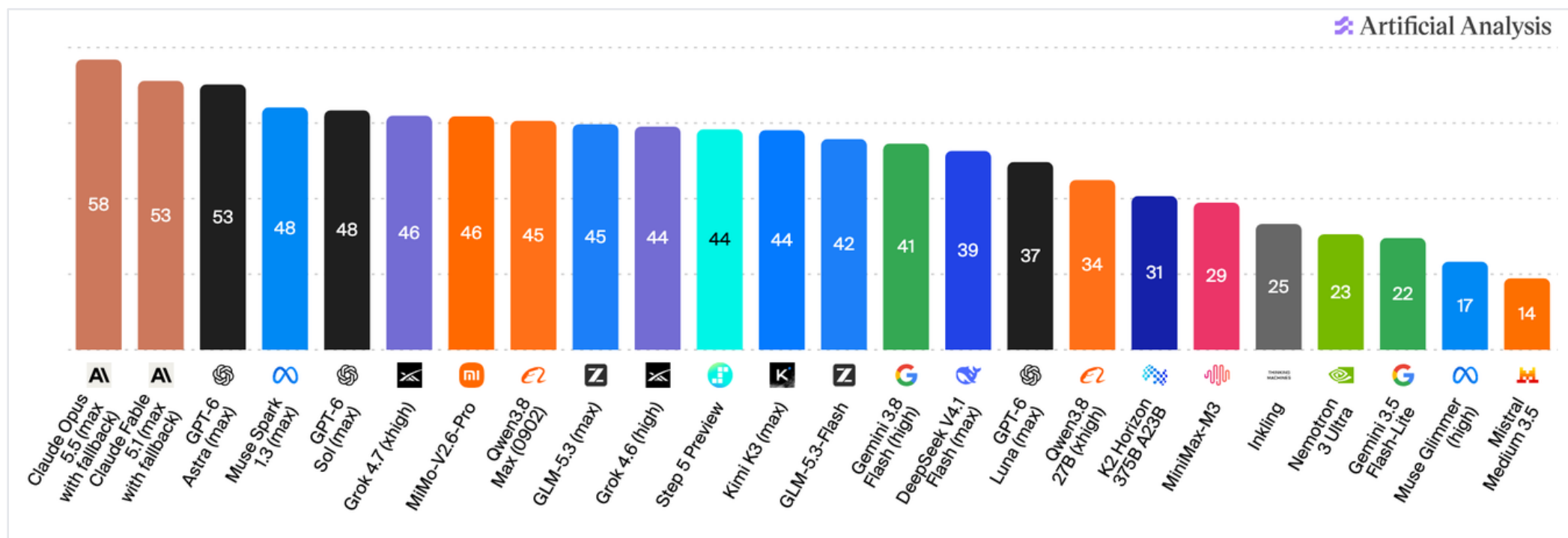
Small models still run locally

gpt-oss-20b on 16 GB · Qwen3-8B on one good GPU

Open or closed, frontier models usually come through an API — open weights just let you pick the host.

Sources: model cards (OpenAI, Meta, DeepSeek, Moonshot); NVIDIA Technical Blog, Aug 2026 (Qwen3.8 on GB300 NVL72); Chapter 0.4.

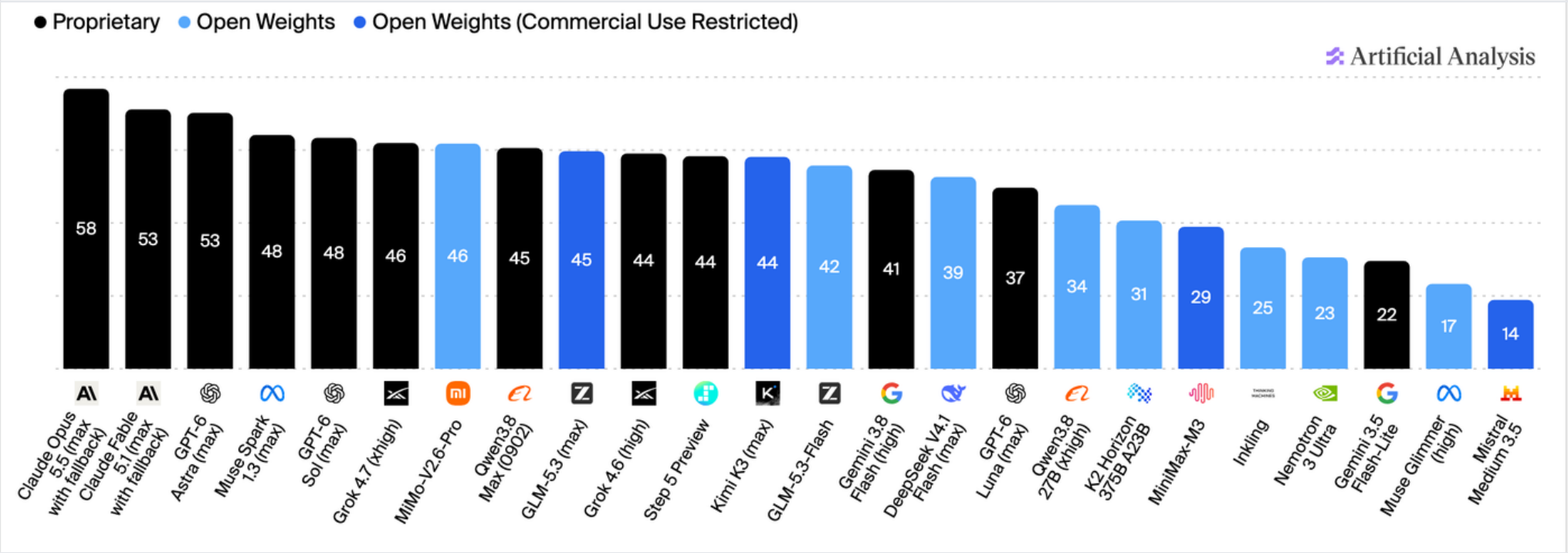
Frontier models today: top 25 of 673, scored on ten tests



Claude Opus 5.5 leads at 58; the top six mix Anthropic, OpenAI, Meta and xAI. Next: which of these can you download?

Artificial Analysis Intelligence Index v4.3.2 (10 evaluations), top 25 of 673 models, captured 25 Sep 2026. Bar colour = developer.

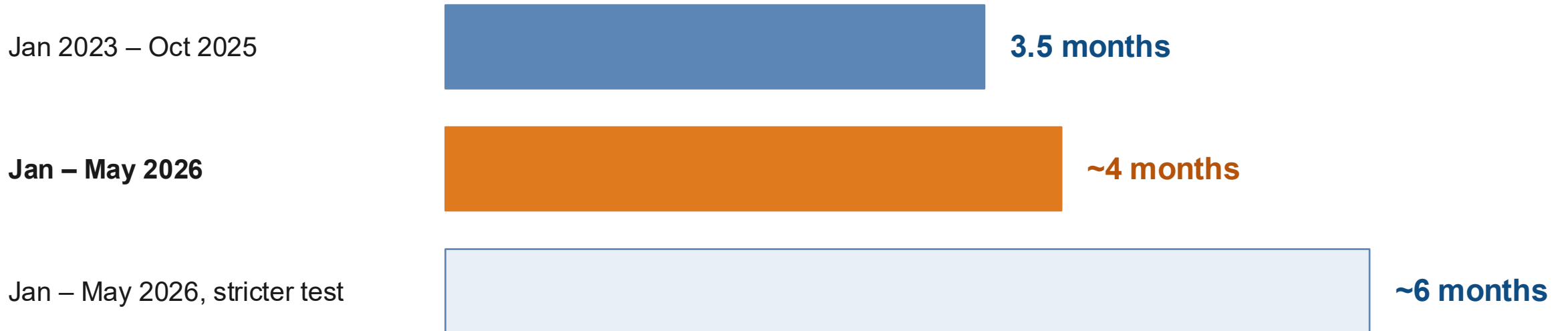
The top six are closed; open weights are close behind



Best open model, MiMo-V2.6-Pro: 46 vs. the leader's 58. GLM-5.3-Flash, served by Lumen: 42.

Artificial Analysis Intelligence Index v4.3.2, top 25 of 673 models, captured 25 Sep 2026.

Open weights trail the closed frontier by about four months

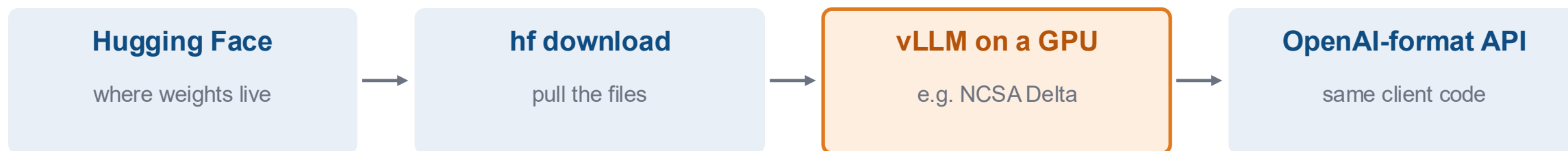


Average lag of the best open-weight model behind the closed frontier (months)

About 8 points on Epoch's Capabilities Index — roughly GPT-5 vs. GPT-5.5.

Epoch AI data insights, Oct 30 2025 and May 29 2026 (epoch.ai/data-insights).

Open weights mean campus GPUs can serve a model themselves



```
pip install vllm "huggingface_hub[cli]"
hf download Qwen/Qwen3-8B
vllm serve Qwen/Qwen3-8B --port 8000
```

Not needed for the workshop: Lumen already serves GLM-5.3-Flash for you.

Delta access comes through an allocation, e.g. Illinois Computes. Only the `base_url` changes in your client code.

Open or closed? Pick one for each task, with a reason

1

A memo quoting a client's private holdings

Open

Closed

2

A summary of public news about Deere

Open

Closed

3

A first draft of a Python script for the data team

Open

Closed

Weigh where the data goes, what it costs, and how strong the model must be.

CHAPTER

From model to agent: tools, MCP and skills

Everything so far is one model call. Part I wraps it in a loop and hands it tools.

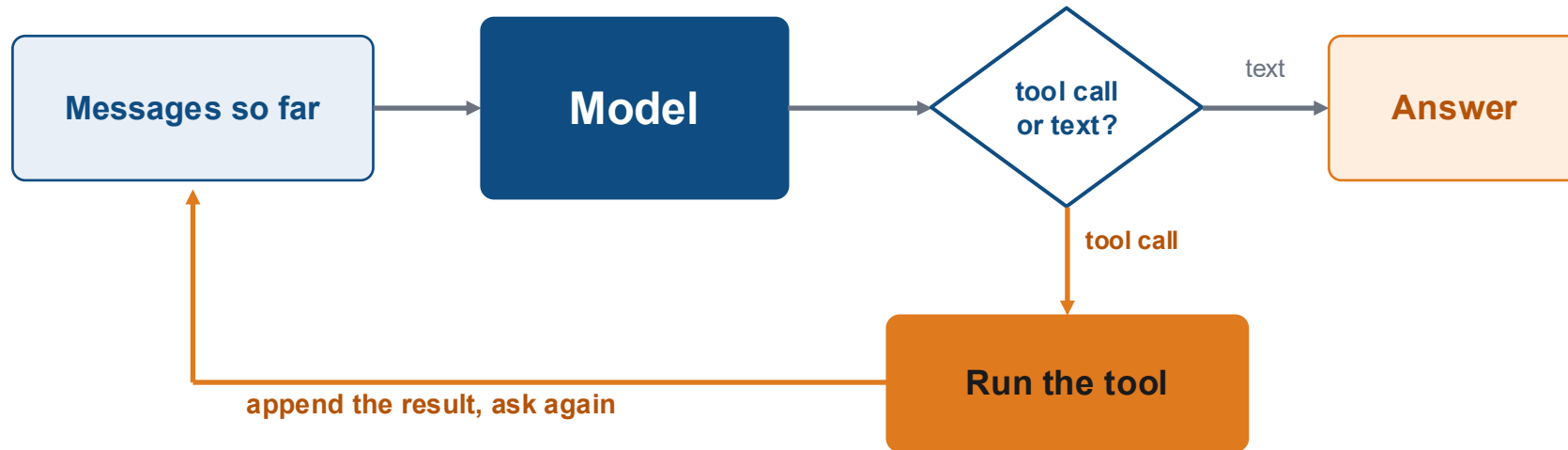
An agent is a model that decides its own next step

WHO DECIDES THE STEPS

Single model call <i>"Summarise this earnings call."</i>	Nobody — there's one step	Model
Workflow <i>Extract vendor → classify invoice → draft email</i>	Your code, in advance	Step 1 → Step 2 → Step 3
Agent <i>"Compare Deere's growth with Caterpillar's."</i>	The model, as it goes	Model → tool? → Model → tool? → Answer

A workflow is a recipe; an agent is a cook. Use a recipe when you can write the steps down.

Every agent is a model, some tools, and a loop



Model

Reads the messages; returns a tool request or an answer. No memory.

Tools

Name + description + argument schema. Your code runs them.

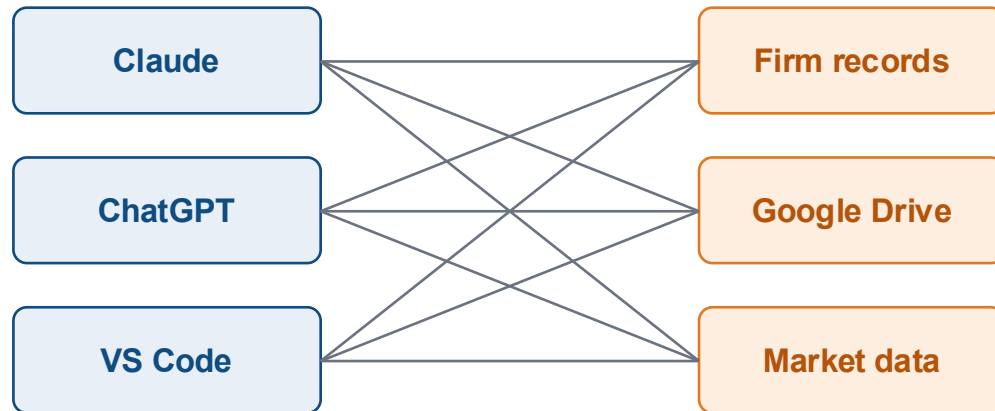
Loop

Runs tools, appends results, enforces step budget and approvals.

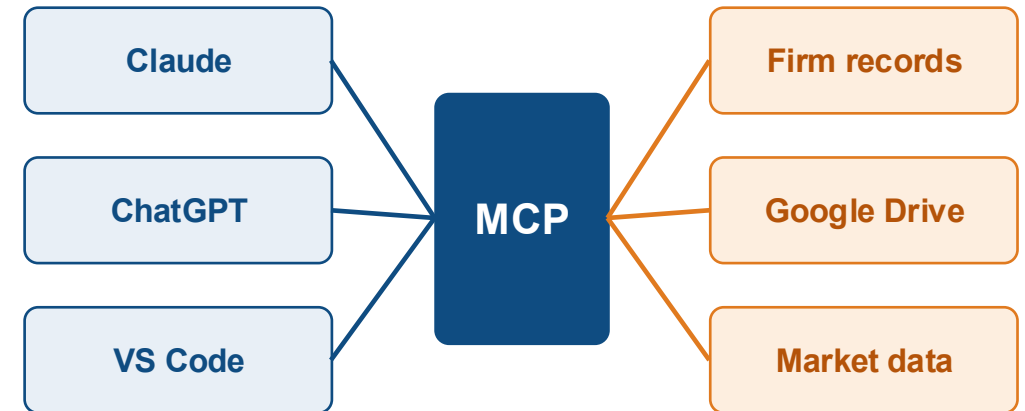
The model never runs code — it only asks for a tool by name. The loop holds the keys.

MCP is a standard plug that connects any AI app to any tool

Without a standard: $3 \times 3 = 9$ custom connectors



With MCP: $3 + 3 = 6$, build each side once



An MCP server can offer tools (actions), resources (data) and prompts (templates). Introduced by Anthropic in Nov 2024; now open and supported by Claude, ChatGPT, VS Code, Cursor and more.

Write the firm's `get_financials` once as an MCP server — every agent and app can use it.

Source: modelcontextprotocol.io ("like a USB-C port for AI applications"). App and server names are examples.

A skill is a playbook the agent loads only when it needs it

client-memo/

SKILL.md

name, description, the steps

house-style.md

firm tone and table rules

check_figures.py

a script the agent can run

template.docx

a starting document

PROGRESSIVE DISCLOSURE · what enters the context window

1 Always

Name + one-line description of every skill

2 When relevant

The full SKILL.md is read

3 Only as needed

Extra files and scripts

Dozens of skills cost only a line each until one is used.

Tools let an agent act; skills teach it how your firm does the job.

Source: Anthropic, “Equipping agents for the real world with Agent Skills” (Oct 16, 2025). The client-memo skill is an example.

Tools act, MCP connects, skills teach — all share one window

Tool

One action the model can ask for

```
get_financials(ticker)
```

Like a calculator

MCP

The standard way to plug tools and data into any app

```
Firm-records MCP server
```

Like the socket

Skill

Know-how: steps, style, scripts for a type of task

```
client-memo/ SKILL.md
```

Like the playbook

Every tool description and skill summary is read on every call — choose what's on the desk (Ch 2).

Five things to take into Part I

01

An LLM predicts the next token
— one skill, repeated.

02

Fluent isn't true: every figure
cites its source.

03

Attention links tokens; MoE runs
a few experts each.

04

Training rewards what scores
well, not what's true.

05

Open weights buy control;
closed models still lead.

**Next: 0D, What an API is
then Chapter 1 — AI Agents**

Questions?

GO DEEPER

Workshop textbook, Part 0: chapters 0, 0A, 0B, 0C — then 0D, APIs

Vaswani et al., Attention Is All You Need (2017) · DeepSeek-V3 Technical Report (2024)

OpenAI gpt-oss model card (2025) · Epoch AI, open vs. closed gap (2025, 2026)

Artificial Analysis, artificialanalysis.ai/models · 3Blue1Brown, Transformers

MSBA AI Workshop · Next: Chapter 1, Introduction to AI Agents